

CHAPTER 1

INTRODUCTION

1.1. OVERVIEW

Primary healthcare is the first contact a person has with the health system when they have a health problem. Primary healthcare refers to a broad range of health services provided by medical professionals in the community. Your general practitioner (GP) is a primary healthcare provider, and so are nurses, pharmacists and allied health providers like dentists. Primary healthcare is the provision of health services, including diagnosis and treatment of a health condition, and support in managing long-term healthcare, including chronic conditions like diabetes. Primary healthcare includes seeing health professionals to help you maintain good health, with regular health checks, health advice when you have concerns, and support for ongoing care.



Primary healthcare is a term used to describe a range of healthcare providers who work in the community. Any healthcare professional who is the first point of contact for the health system can be a primary healthcare provider. Most people visit their GP (sometimes referred to as the 'local doctor') as a first step when they have a health problem that is not an emergency. The primary healthcare system also includes allied health professionals, such as dentists and physiotherapists. Primary healthcare is provided in a community setting, such as a general practice or dental clinic. Primary healthcare providers may work in

medical clinics, community health centres and allied health practices, such as physiotherapy and podiatry practices.

Primary healthcare services

Services delivered by primary healthcare providers include:

- diagnosis, treatment and care of people with health problems
- promoting good health
- preventing health problems
- early intervention
- Managing ongoing and long-term conditions.

Healthcare services play an important role in advising and helping you to prevent illness and maintain good health. Important areas that you should seek advice on include:

- managing and stopping smoking
- hazardous drinking
- stress and depression
- poor diet and
- Physical inactivity.

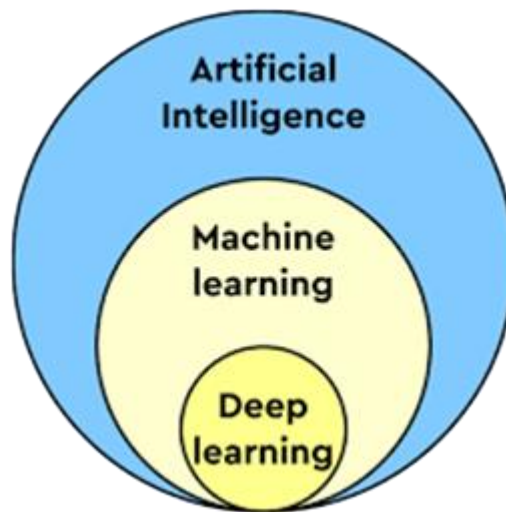
1.2. PROBLEM STATEMENT

Primary Health Centers (PHCs) serve as the backbone of healthcare delivery in rural and semi-urban regions, providing essential medical services such as outpatient care, maternal health support, disease monitoring, and emergency treatment. However, one of the major challenges affecting the efficiency of PHCs is the irregular attendance of doctors during official duty hours. Many healthcare centers still depend on manual attendance registers or basic biometric systems that fail to ensure the actual presence of doctors throughout the working day. These traditional methods are highly vulnerable to proxy attendance, manipulation, and lack real-time validation. Due to the absence of continuous monitoring mechanisms, health authorities often have limited visibility into whether doctors are present at their assigned locations. Cases of late arrivals, early departures, or complete absenteeism are typically identified only after patient complaints or periodic inspections, leading to delayed corrective actions. This lack of accountability directly impacts patient care, resulting in long waiting times, untreated medical conditions, and reduced public trust in government healthcare services. Furthermore, existing systems do not support geolocation-based verification or intelligent tracking of duty hours, making it impossible to confirm whether doctors remain within PHC premises during working hours. There is also no automated mechanism for alert generation,

compliance monitoring, or performance analysis. The absence of real-time monitoring, reliable identity verification, and automated reporting creates a critical gap in healthcare administration. To address these challenges, there is a strong need for an intelligent, automated, and secure attendance monitoring system that integrates facial recognition, geo-location validation, and continuous duty-hour tracking. Such a system can ensure transparency, improve accountability, and enhance the overall efficiency of rural healthcare delivery.

1.3. DEEP LEARNING

Deep learning is a method in artificial intelligence (AI) that teaches computers to process data in a way that is inspired by the human brain. Deep learning models can recognize complex patterns in pictures, text, sounds, and other data to produce accurate insights and predictions.



Deep learning algorithms are neural networks that are modeled after the human brain. For example, a human brain contains millions of interconnected neurons that work together to learn and process information. Similarly, deep learning neural networks, or artificial neural networks, are made of many layers of artificial neurons that work together inside the computer. Artificial neurons are software modules called nodes, which use mathematical calculations to process data. Artificial neural networks are deep learning algorithms that use these nodes to solve complex problems.

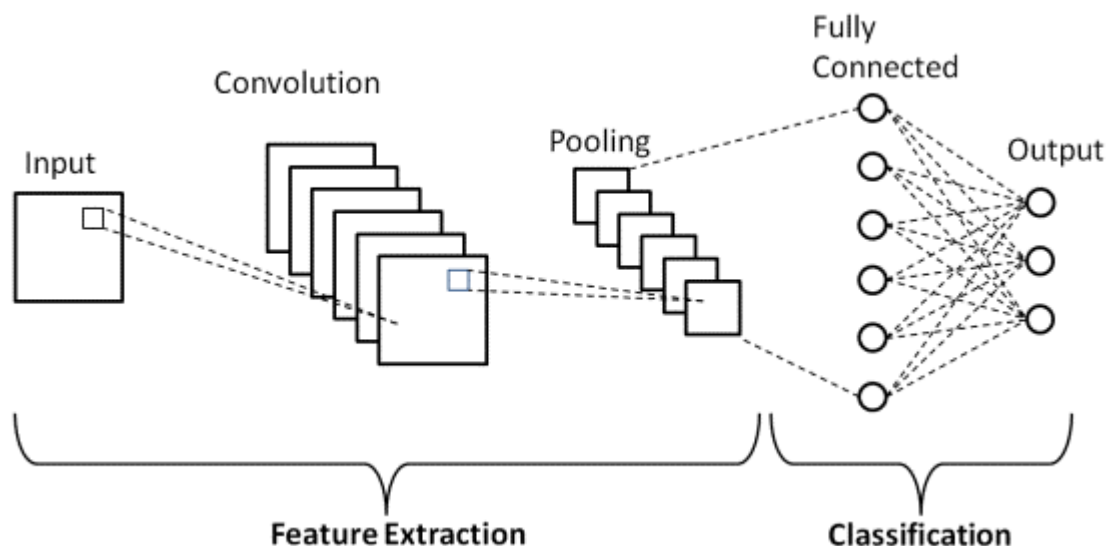
1.3.1. Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a type of deep learning neural network architecture that is particularly well suited to image classification and object recognition tasks. A CNN works by transforming an input image into a feature map, which is then processed through multiple convolutional and

pooling layers to produce a predicted output. In this blog post, we will explore the basics of CNNs, including how they work, their architecture, and how they can be used for a wide range of computer vision tasks. We will also provide examples of some real-world applications of CNNs, and outline some of the benefits and limitations of this deep-learning architecture. A convolutional neural network starts by taking an input image, which is then transformed into a feature map through a series of convolutional and pooling layers. The convolutional layer applies a set of filters to the input image, each filter producing a feature map that highlights a specific aspect of the input image. The pooling layer then downsamples the feature map to reduce its size, while retaining the most important information.

The feature map produced by the convolutional layer is then passed through multiple additional convolutional and pooling layers, each layer learning increasingly complex features of the input image. The final output of the network is a predicted class label or probability score for each class, depending on the task. A typical CNN architecture is made up of three main components: the input layer, the hidden layers, and the output layer. The input layer receives the input image and passes it to the hidden layers, which are made up of multiple convolutional and pooling layers. The output layer provides the predicted class label or probability scores for each class.

The hidden layers are the most important part of a CNN, and the number of hidden layers and the number of filters in each layer can be adjusted to optimize the network's performance. A common architecture for a CNN is to have multiple convolutional layers, followed by one or more pooling layers, and then a fully connected layer that provides the final output.



Applications of Convolutional Neural Network:

CNNs have a wide range of applications in computer vision, including image classification, object detection, semantic segmentation, and style transfer.

Image classification: Image classification is the task of assigning a class label to an input image. CNNs can be trained on large datasets of labeled images to learn the relationships between the image pixels and the class labels, and then applied to new, unseen images to make a prediction.

Object detection: Object detection is the task of identifying objects of a specific class in an input image and marking their locations. This can be useful for applications such as security and surveillance, where it is important to detect and track objects in real time.

Semantic segmentation: Semantic segmentation is the task of assigning a class label to each pixel in an input image, producing a segmented image that can be used for further analysis. This can be useful for applications such as medical image analysis, where it is important to segment specific structures in an image for further analysis.

Style transfer: Style transfer is the task of transferring the style of one image to another image while preserving the content of the target image. This can be useful for applications such as art and design, where it is desired to create an image that combines the content of one image with the style of another.

1.3.2. GEOFENCING

Geofencing is a type of location-based marketing and advertising. A mobile app or software uses the Global Positioning System (GPS), radio frequency identification (RFID), Wi-Fi or cellular data to define a virtual geographical boundary and trigger a targeted marketing action when a device enters or exits that boundary. This boundary is known as a geofence.



Geofencing, sometimes spelled geo-fencing, can also allow a user to set up triggers so that when a device such as an internet-connected smartphone enters a defined geographical boundary, the user gets an alert. They can also initiate some specific action targeting that device. An example of geofencing used in marketing is when a student and a working professional walk near a computer store, and then both receive a text message that says, "Today only! Buy a laptop and get an e-reader free!" The users get the text because they are near a store that has a geofence that automatically triggers a targeted marketing action -- the text

message. The store could also geotarget users with in-app notifications, social media ads, push notifications, emails, etc. The virtual boundaries or geofences can be active or passive. Active geofences require an end user to opt in to location services and a mobile app to be open. Passive geofences are always on; they rely on Wi-Fi and cellular data, and work in the background. Geofencing requires communication technologies such as GPS, RFID, Wi-Fi or cellular data.

Once a marketer sets up a geofence, a preprogrammed action is automatically triggered when a mobile device or RFID tag enters or exits the geofence. A virtual boundary can be created around a geographical location as small as a building, store or mall, and as large as a ZIP code, city or entire state.



Geofencing enables marketers to target prospects with engaging messages and attractive offers. It can improve customer engagement because geofencing can target a specific demographic within a defined geographical area that's further filtered using targeted criteria.. This includes data about foot traffic, stay durations, conversions, etc. Marketers can combine this data with information about users' online activities, purchase habits and web browsing behaviors to personalize campaigns and improve user experiences. Another advantage of geofencing is that it allows businesses to create omnichannel marketing campaigns to further boost sales and revenues. In addition to marketing, geofencing has other applications because it is mobile-centric. Any company can deploy geofencing on a range of mobile devices, including smartphones, tablets and smartwatches, for marketing, advertising, tracking, security management, law enforcement and more.

1.4. OBJECTIVE OF THE PROJECT

Aim

The aim of this project is to design and develop an AI-enabled attendance and monitoring system that ensures verified doctor identity, continuous location compliance, and improved accountability in

Primary Health Centers. The system aims to enhance transparency, reduce absenteeism, and improve the overall quality and reliability of healthcare services in rural areas through intelligent automation.

Objectives

- To design and implement a secure AI-enabled attendance monitoring system for PHC doctors.
- To authenticate doctors using credential-based login and biometric verification.
- To apply facial recognition techniques for accurate identity validation.
- To capture real-time GPS coordinates to confirm physical presence at PHC locations.
- To enforce geofencing mechanisms to restrict attendance marking within authorized zones.
- To perform random interval face re-verification to prevent proxy attendance.
- To continuously monitor duty-hour location and movement of doctors.
- To automatically detect absenteeism, early departures, and geofence violations.
- To generate real-time alerts and notifications for administrators and health officials.
- To record and manage patient consultation data for service monitoring.
- To generate analytical reports on attendance trends and compliance behavior.
- To improve transparency, accountability, and efficiency in rural healthcare delivery.

1.5. SCOPE OF THE PROJECT

The project is designed to enhance accountability, transparency, and efficiency in doctor attendance management at Primary Health Centers (PHCs). It leverages AI-based facial recognition, geotagged attendance verification, continuous GPS tracking, and random interval re-verification to ensure accurate monitoring of duty-hour presence.

The scope of the system includes the following key areas:

- **Automated Attendance Monitoring:** Real-time recording of doctor attendance with location verification to prevent fraudulent entries.
- **Facial Recognition for Identity Verification:** Uses CNN-based recognition to authenticate doctors and eliminate proxy attendance.
- **Random Interval Re-Verification:** Periodically re-checks doctor identity and location during duty hours to ensure continuous presence.
- **Geofence-Based Location Validation:** Ensures attendance is marked only within authorized PHC boundaries.
- **Absenteeism Alerts and Escalation:** Sends automatic notifications to doctors, PHC administrators, and higher authorities for non-compliance.

- **Patient Activity Logging:** Allows doctors to record patient consultations, treatments, and medical details for enhanced service tracking.
- **Real-Time Monitoring and Reporting:** Centralized dashboard for administrators to monitor attendance, violations, and generate analytical reports.
- **Policy Enforcement and Decision Support:** Provides insights and analytics to help authorities enforce rules and take corrective action.
- **Scalability and Future Expansion:** Adaptable to multiple PHCs and integrable with broader health management systems for wider implementation.

This project aims to streamline doctor attendance tracking, improve healthcare service delivery, and strengthen administrative oversight in rural and semi-urban PHCs.

CHAPTER 2

SYSTEM STUDY AND ANALYSIS

2.1. EXISTING SYSTEM

Manual Attendance Registers

Most Primary Health Centers maintain paper-based attendance books where doctors sign in at the beginning and out at the end of their shifts. These registers are reviewed periodically by administrative staff. While this method is simple and inexpensive, it is highly prone to human errors, such as misentries or missing signatures. It is also vulnerable to intentional manipulation, where someone could sign on behalf of an absent doctor. Moreover, manual registers do not provide real-time verification, making it difficult for administrators to monitor doctor presence accurately.

Periodic Manual Inspections

In some PHCs, supervisors or administrators conduct periodic visits to physically check whether doctors are present. While inspections provide a layer of accountability, they are infrequent and cannot guarantee consistent monitoring. High patient loads or large PHC networks make it impractical for supervisors to verify every doctor regularly. Furthermore, the process is reactive rather than proactive, as absenteeism is often detected only after complaints or audits.

Fingerprint / Biometric Devices

Fingerprint scanners, thumb-scan devices, and other biometric systems are used in some PHCs to automate attendance tracking. These systems record the time of check-in and check-out, reducing the chances of proxy attendance compared to manual registers. However, they still do not verify the doctor's location or presence during duty hours. Technical issues such as device malfunctions, poor-quality fingerprint capture, or connectivity problems can affect reliability. Maintenance costs and the need for technical support are additional limitations.

RFID or Smart Card Systems

Some PHCs issue smart ID cards or RFID cards to doctors for attendance marking. Doctors swipe the card when arriving or leaving the facility. These systems are faster and more automated than manual registers, but they cannot ensure that the doctor remains physically present within the PHC during duty hours. Cards can be misused if swiped by another person, and they lack continuous monitoring capabilities.

2.1.1. DISADVANTAGES

- Proxy attendance is possible.
- No continuous monitoring of doctors.
- Manual errors in attendance records.
- Location not verified in biometric/RFID systems.
- Periodic inspections are inconsistent.
- Absenteeism detected late.
- Biometric devices may fail.
- No integration with patient data.
- Reduced accountability in PHCs.

2.2. PROPOSED SYSTEM

The **proposed system** is an AI-enabled attendance and monitoring platform designed for Primary Health Centers (PHCs) to ensure verified doctor identity, continuous location compliance, and improved accountability during duty hours. It integrates advanced facial recognition, geolocation verification, and real-time monitoring to address the limitations of existing attendance methods. The system consists of the following key components:

CNN-Based Facial Identity Verification

The system employs a Convolutional Neural Network (CNN) to extract distinctive facial features from live images captured by doctors when marking attendance. These features are compared with pre-registered templates to authenticate the identity, eliminating proxy attendance and ensuring accurate verification.

Geofencing and Location Validation

A geofencing algorithm defines a secure virtual boundary around each PHC. The system validates the GPS coordinates embedded in the attendance submission to ensure that doctors are physically present within the authorized PHC premises when marking attendance.

Continuous Location Monitoring

The system periodically collects GPS updates at fixed intervals (typically every 10–15 minutes) to monitor doctors' movement during duty hours. This ensures that doctors remain within the PHC boundaries and allows detection of unusual behavior, prolonged inactivity, or absence during working hours.

Random Interval Geo-Tagged Face Re-Verification

To strengthen compliance, a rule-based scheduling algorithm triggers re-verification at unpredictable time intervals during duty shifts. Doctors are required to submit a live facial image with GPS metadata, which is validated again using the CNN model and geofence check. This prevents absenteeism or unauthorized departures that may occur after the initial check-in.

Violation Detection and Alerting

A rule-driven decision module analyzes identity verification results and location data in real time. It detects absenteeism, geofence breaches, or failed re-verifications and automatically sends alerts to PHC administrators, block-level authorities, and higher officials, enabling timely corrective action.

Patient Activity Logging

Doctors can log daily patient consultations through the system. Each entry is timestamped and stored, allowing administrators to monitor service delivery and correlate patient activity with attendance and presence compliance. This ensures accountability not only for attendance but also for healthcare services provided.

2.2.1. Benefits of the Proposed System

- Ensures real-time verification of doctor presence.
- Reduces absenteeism and unauthorized duty leave.
- Improves accountability and transparency in PHCs.
- Provides accurate, timestamped attendance records.
- Enables timely alerts and corrective actions for authorities.
- Supports monitoring of service delivery through patient logs.
- Facilitates data-driven decision-making for health administrators.
- Minimizes manual effort and paperwork in attendance management.
- Enhances overall efficiency and reliability of rural healthcare services.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1. HARDWARE REQUIREMENTS

- **Processor** : Intel Core i5 or higher
- **RAM** : 8GB or above
- **Storage** : 250GB SSD or more
- **Camera** : HD Webcam / Smartphone Camera for face recognition
- **GPS Module** : For Geo-tagged verification
- **Internet** : Stable connection for real-time data transfer
- **Backup Storage** : External HDD / Cloud storage for data safety
- **Power Supply / UPS** : To ensure uninterrupted system operation

3.2. SOFTWARE REQUIREMENTS

- **Operating System** : Windows 10/11
- **Backend** : Python, Flask
- **Frontend** : HTML, CSS, JavaScript, Bootstrap, React.js
- **Database** : MySQL
- **Machine Learning** : TensorFlow, Keras, OpenCV, NumPy for CNN-based face recognition
- **Geo-location API** : Google Maps API for GPS and geofence validation
- **Web Server** : Apache HTTP Server / WampServer

3.3. SOFTWARE DESCRIPTION

3.3.1. PYTHON 3.7.4

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. It was created by Guido van Rossum during 1985- 1990. Like Perl, Python source code is also available under the GNU General Public License (GPL). This tutorial gives enough understanding on Python programming language.



Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages. Python is a MUST for students and working professionals to become a great Software Engineer especially when they are working in Web Development Domain.

Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc. The biggest strength of Python is huge collection of standard libraries which can be used for the following:

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Scientific computing
- Text processing and many more

Tensor Flow

TensorFlow is an end-to-end open-source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and gives developers the ability to easily build and deploy ML-powered applications.



TensorFlow provides a collection of workflows with intuitive, high-level APIs for both beginners and experts to create machine learning models in numerous languages. Developers have the option to deploy models on a number of platforms such as on servers, in the cloud, on mobile and edge devices, in browsers, and on many other JavaScript platforms. This enables developers to go from model building and training to deployment much more easily.

Keras

Keras is a deep learning API written in Python, running on top of the machine learning platform TensorFlow. It was developed with a focus on enabling fast experimentation.



- Allows the same code to run on CPU or on GPU, seamlessly.
- User-friendly API which makes it easy to quickly prototype deep learning models.
- Built-in support for convolutional networks (for computer vision), recurrent networks (for sequence processing), and any combination of both.
- Supports arbitrary network architectures: multi-input or multi-output models, layer sharing, model sharing, etc. This means that Keras is appropriate for building essentially any deep learning model, from a memory network to a neural Turing machine.

Pandas

Pandas are a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language. Pandas are a Python package that provides fast, flexible,

and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python.



Pandas is mainly used for data analysis and associated manipulation of tabular data in Data frames. Pandas allows importing data from various file formats such as comma-separated values, JSON, Parquet, SQL database tables or queries, and Microsoft Excel. Pandas allows various data manipulation operations such as merging, reshaping, selecting, as well as data cleaning, and data wrangling features. The development of pandas introduced into Python many comparable features of working with Data frames that were established in the R programming language. The panda's library is built upon another library NumPy, which is oriented to efficiently working with arrays instead of the features of working on Data frames.

NumPy

NumPy, which stands for Numerical Python, is a library consisting of multidimensional array objects and a collection of routines for processing those arrays. Using NumPy, mathematical and logical operations on arrays can be performed.



NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

Scikit Learn

Scikit-learn is a Python module for machine learning built on top of SciPy and is distributed under the 3-Clause BSD license.



Scikit-learn (formerly scikits. learn and also known as sklearn) is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support-vector machines, random forests, gradient boosting, k-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.

Pillow

Pillow is the friendly PIL fork by Alex Clark and Contributors. PIL is the Python Imaging Library by Fredrik Lundh and Contributors.



Python pillow library is used to image class within it to show the image. The image modules that belong to the pillow package have a few inbuilt functions such as load images or create new images, etc.

OpenCV

OpenCV is an open-source library for the computer vision. It provides the facility to the machine to recognize the faces or objects.



In OpenCV, the CV is an abbreviation form of a computer vision, which is defined as a field of study that helps computers to understand the content of the digital images such as photographs and videos.

3.3.2. MYSQL

MySQL is a relational database management system based on the Structured Query Language, which is the popular language for accessing and managing the records in the database. MySQL is open-source and free software under the GNU license. It is supported by Oracle Company. MySQL database that provides for how to manage database and to manipulate data with the help of various SQL queries. These queries are: insert records, update records, delete records, select records, create tables, drop tables, etc. There are also given MySQL interview questions to help you better understand the MySQL database.



MySQL is currently the most popular database management system software used for managing the relational database. It is open-source database software, which is supported by Oracle Company. It is fast, scalable, and easy to use database management system in comparison with Microsoft SQL Server and Oracle Database. It is commonly used in conjunction with PHP scripts for creating powerful and dynamic server-side or web-based enterprise applications. It is developed, marketed, and supported by MySQL AB, a Swedish company, and written in C programming language and C++ programming language. The official pronunciation of MySQL is not the My Sequel; it is My Ess Que Ell. However, you can pronounce it in your way. Many small and big companies use MySQL. MySQL supports many Operating Systems like Windows, Linux, MacOS, etc. with C, C++, and Java languages.

3.3.3. WAMP SERVER

WampServer is a Windows web development environment. It allows you to create web applications with Apache2, PHP and a MySQL database. Alongside, PhpMyAdmin allows you to manage easily your database.



WAMP Server is a reliable web development software program that lets you create web apps with MYSQL database and PHP Apache2. With an intuitive interface, the application features numerous functionalities and makes it the preferred choice of developers from around the world. The software is free to use and doesn't require a payment or subscription.

3.3.4. BOOTSTRAP 4

Bootstrap is a free and open-source tool collection for creating responsive websites and web applications. It is the most popular HTML, CSS, and JavaScript framework for developing responsive, mobile-first websites.



It solves many problems which we had once, one of which is the cross-browser compatibility issue. Nowadays, the websites are perfect for all the browsers (IE, Firefox, and Chrome) and for all sizes of screens (Desktop, Tablets, Phablets, and Phones). All thanks to Bootstrap developers -Mark Otto and Jacob Thornton of Twitter, though it was later declared to be an open-source project.

Easy to use: Anybody with just basic knowledge of HTML and CSS can start using Bootstrap

Responsive features: Bootstrap's responsive CSS adjusts to phones, tablets, and desktops

Mobile-first approach: In Bootstrap, mobile-first styles are part of the core framework

Browser compatibility: Bootstrap 4 is compatible with all modern browsers (Chrome, Firefox, Internet Explorer 10+, Edge, Safari, and Opera)

3.3.5. FLASK

Flask is a web framework. This means flask provides you with tools, libraries and technologies that allow you to build a web application. This web application can be some web pages, a blog, a wiki or go as big as a web-based calendar application or a commercial website.



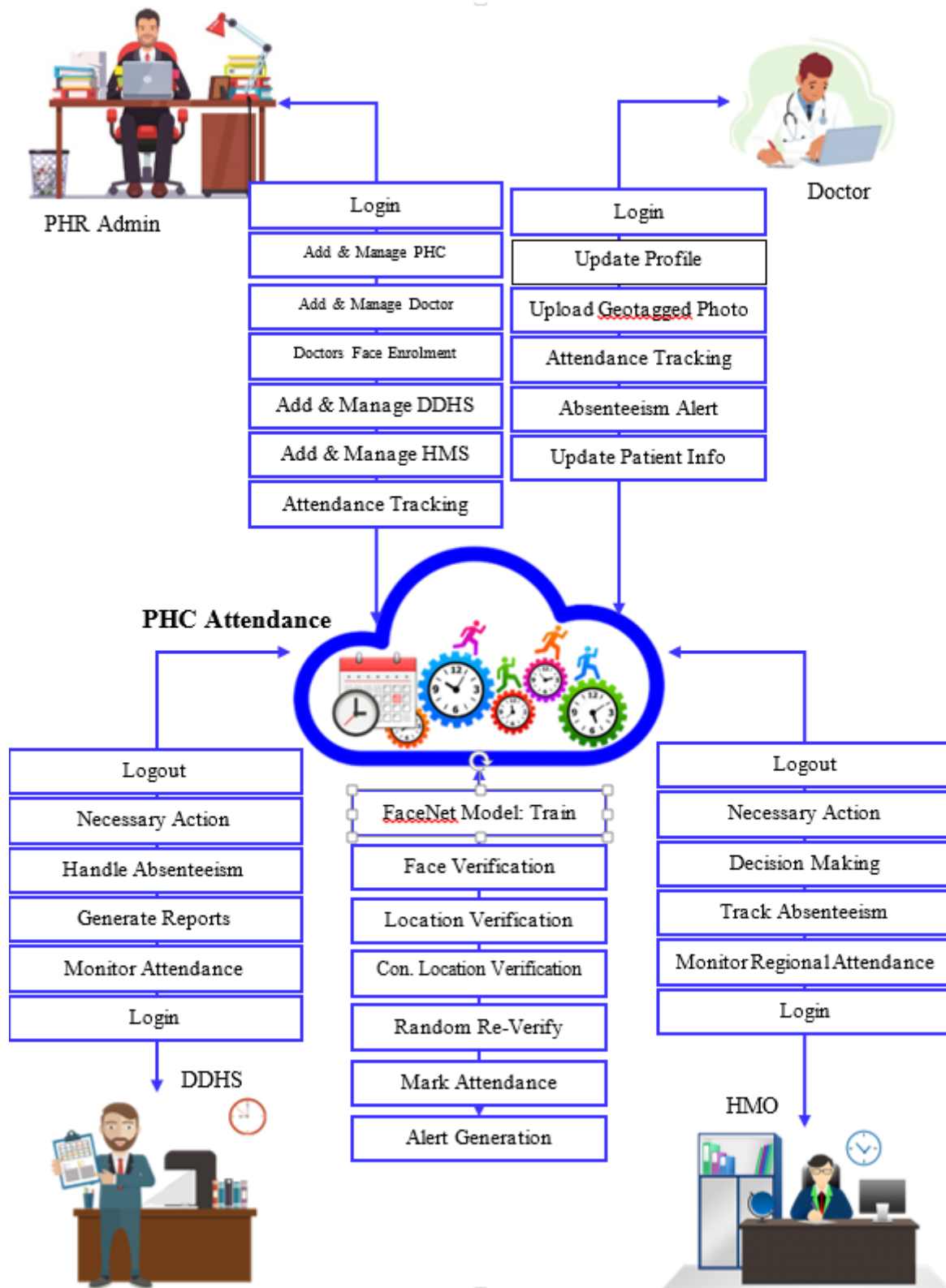
Flask is often referred to as a micro framework. It aims to keep the core of an application simple yet extensible. Flask does not have built-in abstraction layer for database handling, nor does it have formed a validation support. Instead, Flask supports the extensions to add such functionality to the application. Although Flask is rather young compared to most Python frameworks, it holds a great promise and has already gained popularity among Python web developers. Let's take a closer look into Flask, so-

called “micro” framework for Python. Flask is part of the categories of the micro-framework. Micro-framework are normally framework with little to no dependencies to external libraries. This has pros and cons. Pros would be that the framework is light, there are little dependency to update and watch for security bugs, cons is that some time you will have to do more work by yourself or increase yourself the list of dependencies by adding plugins.

CHAPTER 4

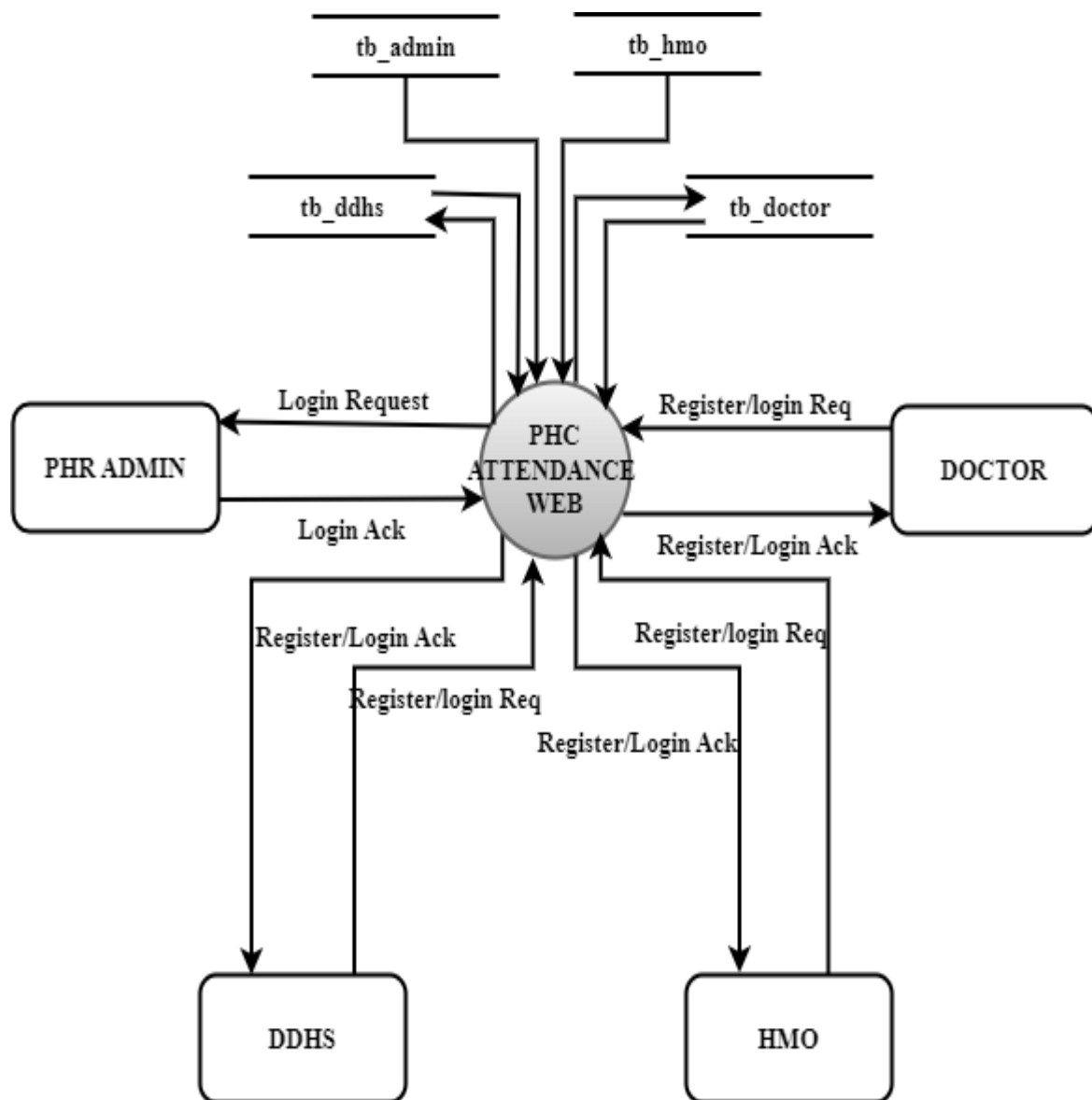
SYSTEM DESIGN

4.1. SYSTEM ARCHITECTURE

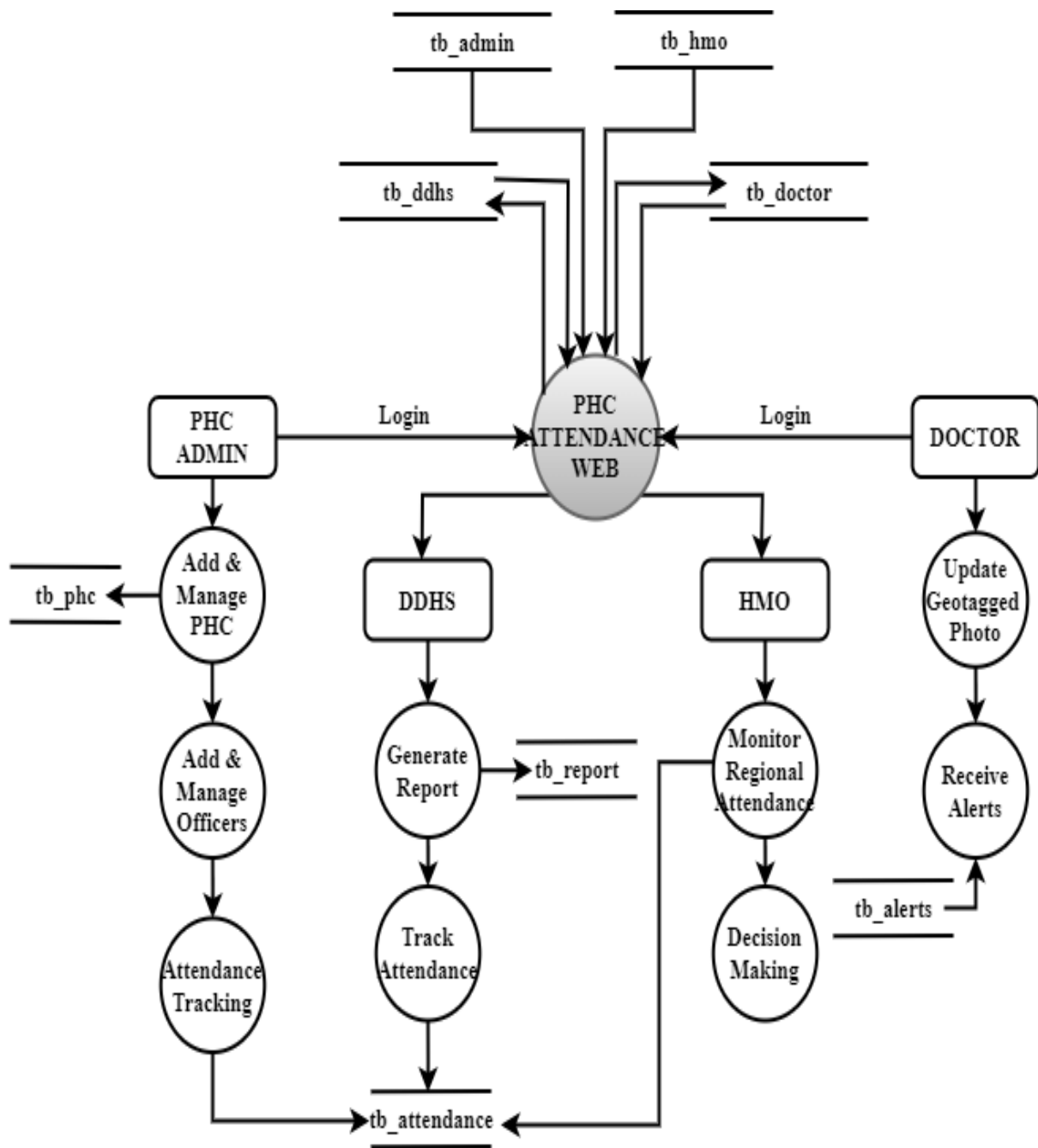


4.2. DATA FLOW DIAGRAM

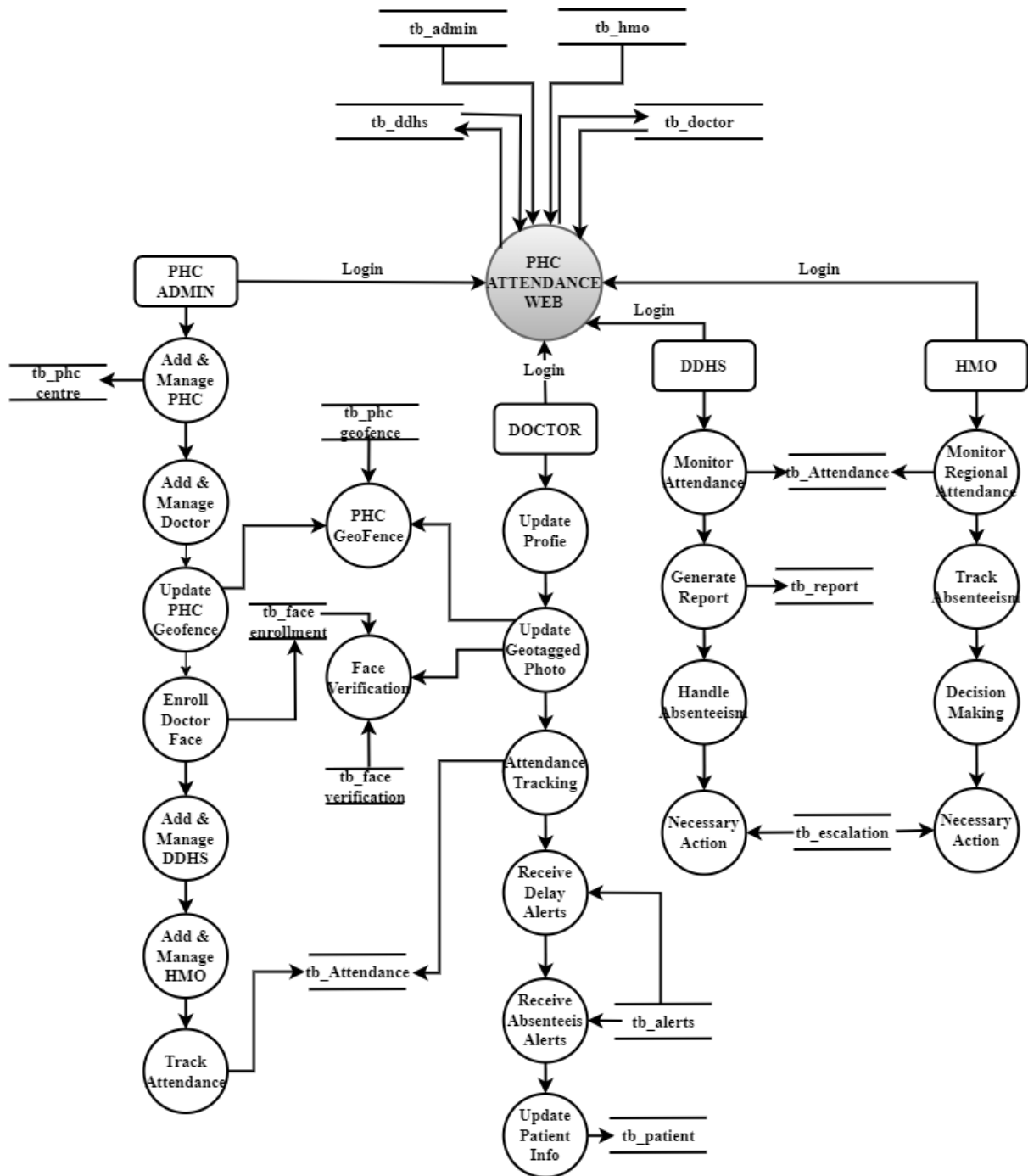
LEVEL 0



LEVEL 1

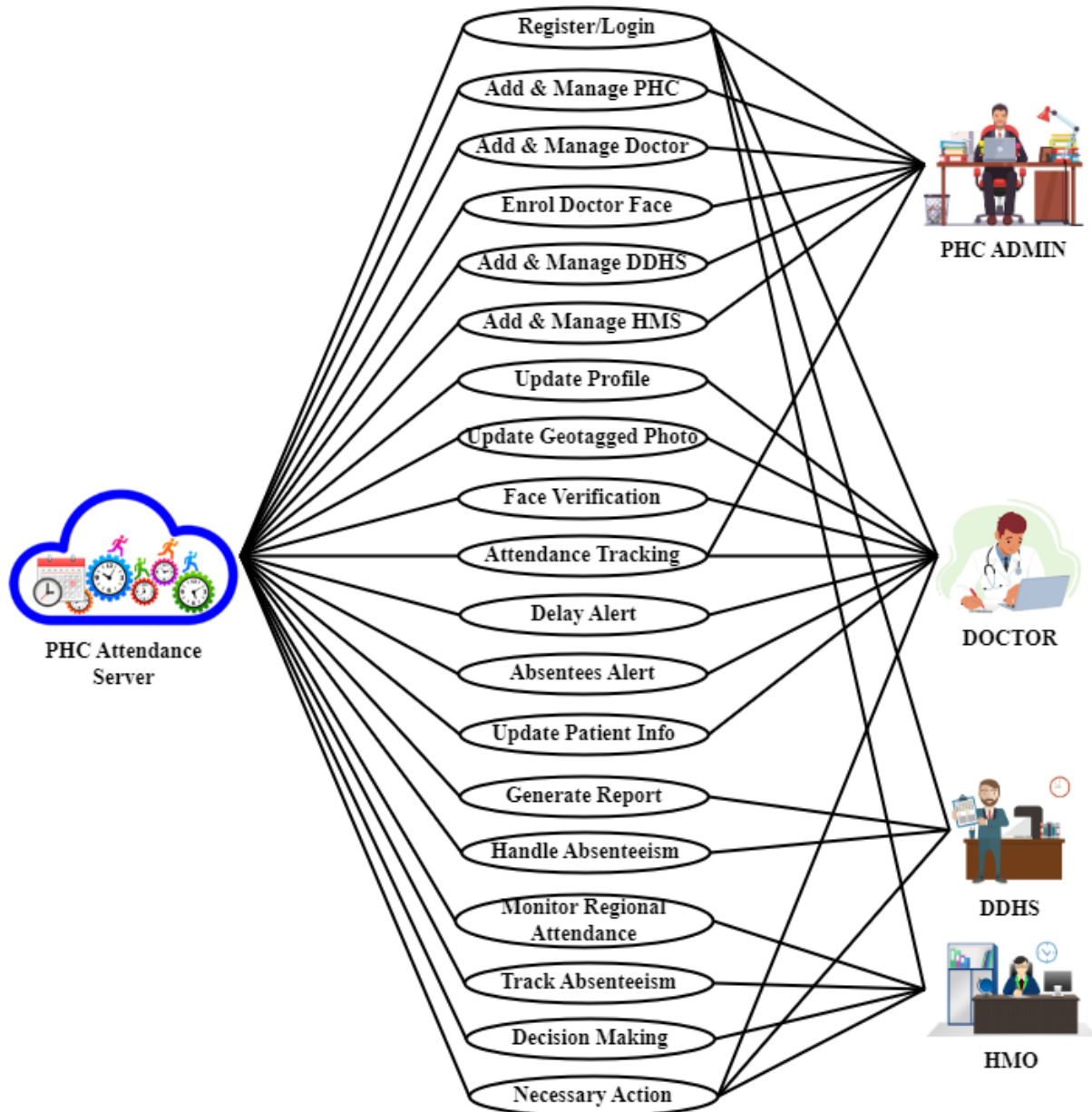


LEVEL 2

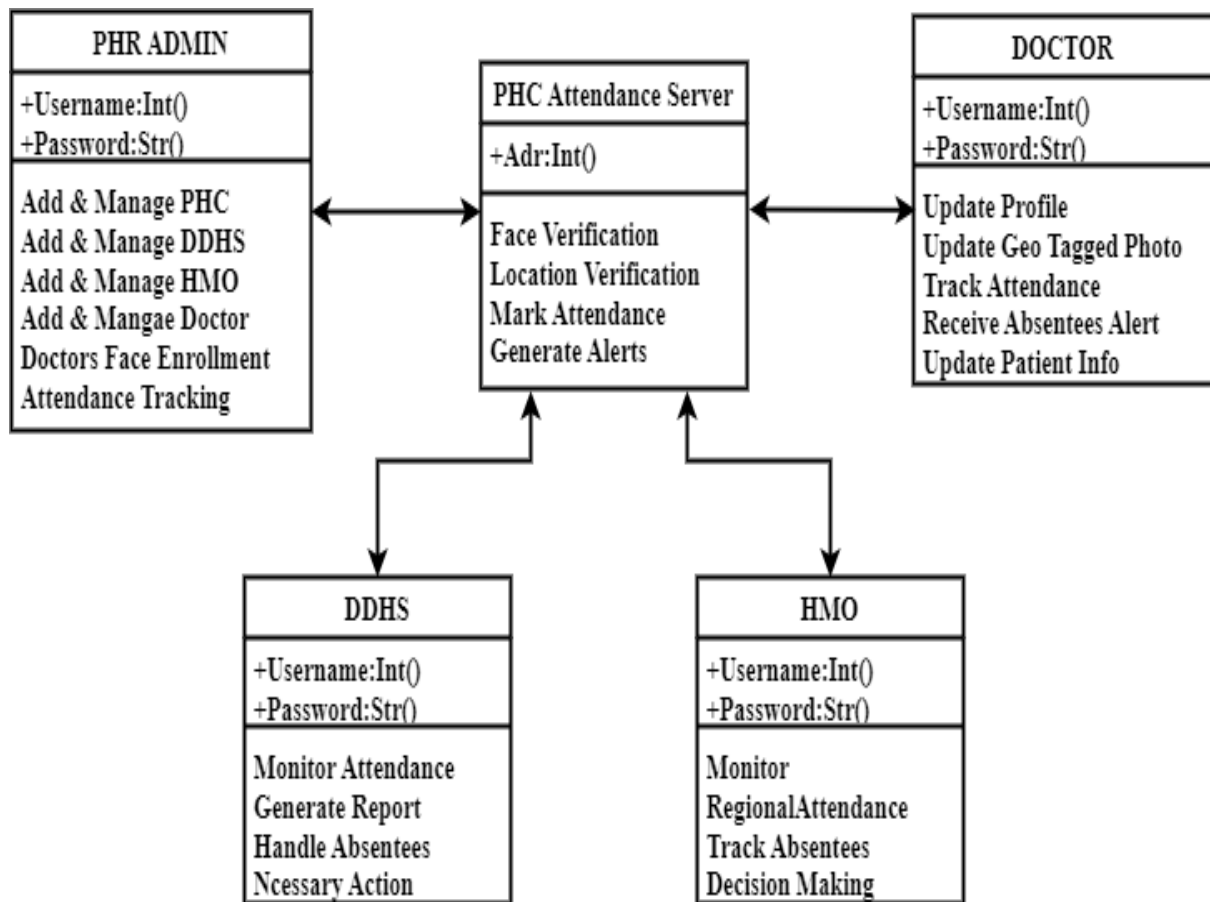


4.3. UML DIAGRAM

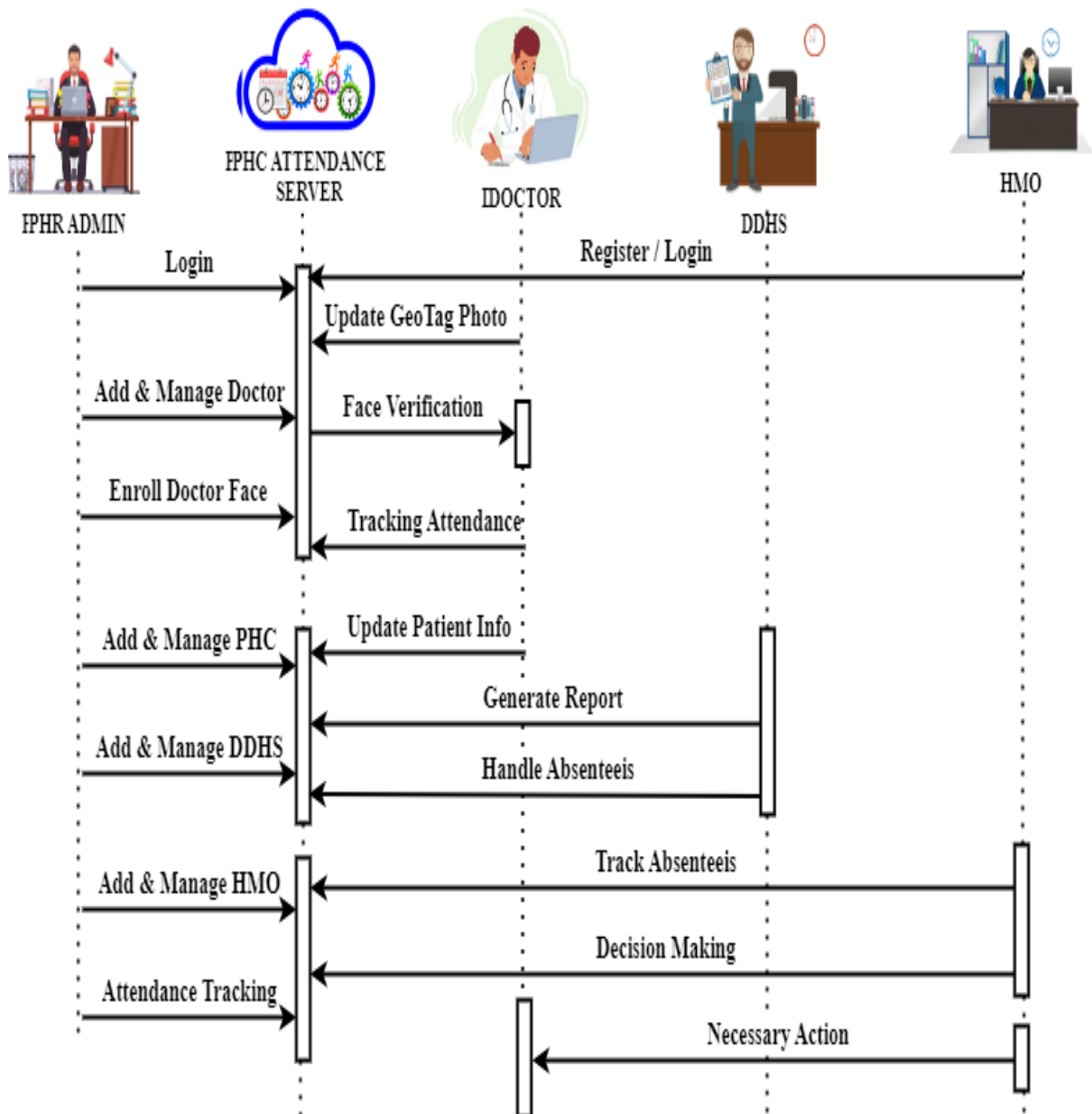
4.3.1. USE CASE DIAGRAM



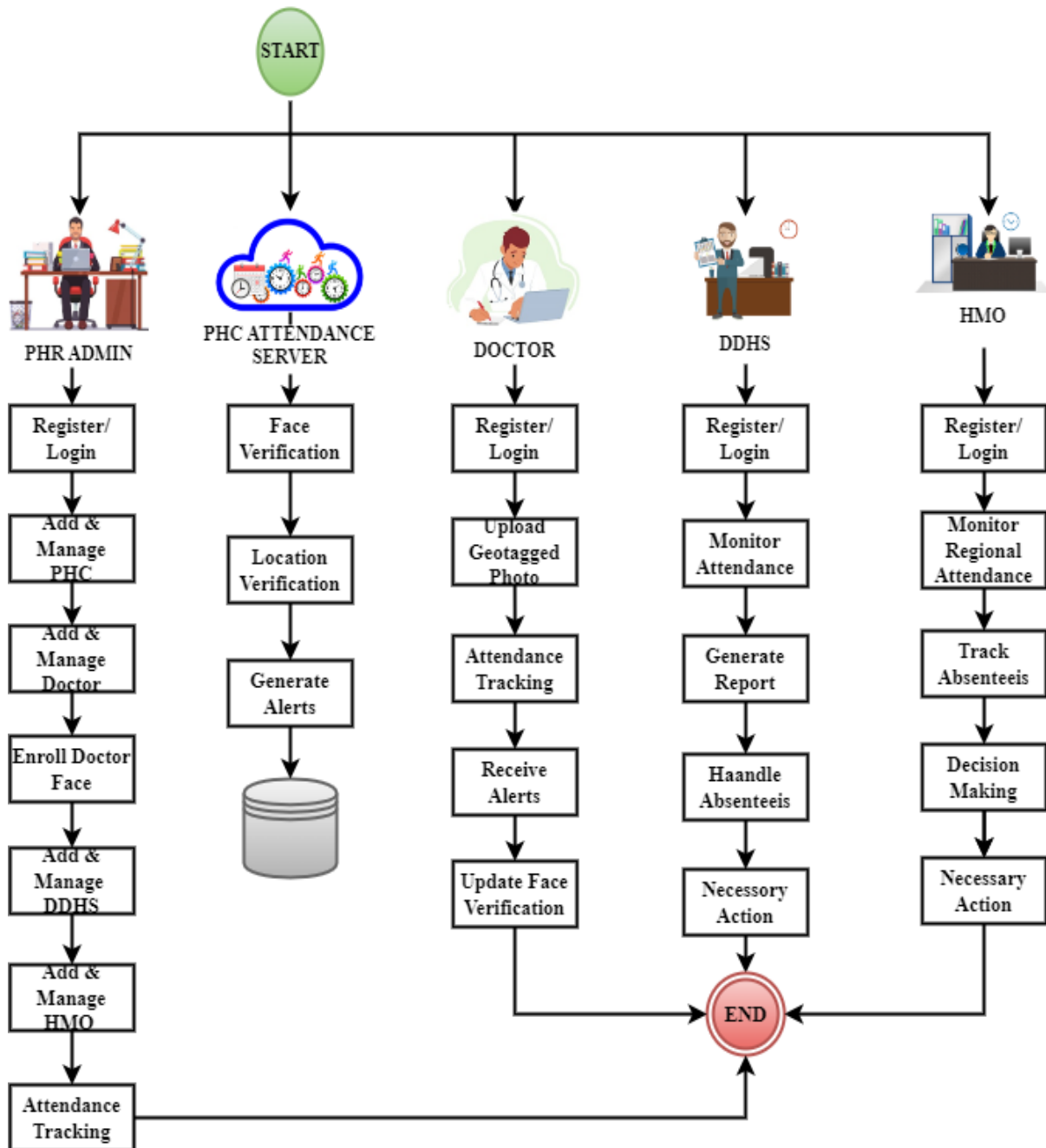
4.3.2. CLASS DIAGRAM



4.3.3. SEQUENCE DIAGRAM



4.3.4. ACTIVITY DIAGRAM



4.4. DATABASE DESIGN

1. Admin Table

Field Name	Data Type (Size, Constraints)	Description
Admin ID	Varchar(10), Primary Key	Unique identifier for the admin
Name	Varchar(50), Not Null	Name of the admin
Email	Varchar(100), Unique, Not Null	Admin's email address
Password	Varchar(100), Not Null	Encrypted password
Role	Varchar(30), Not Null	Role (e.g., Super Admin)
Contact Number	Varchar(15), Not Null	Phone number
Created Date	Date, Not Null	Account creation date

2. Doctor Table

Field Name	Data Type (Size, Constraints)	Description
Doctor ID	Varchar(10), Primary Key	Unique ID of the doctor
Name	Varchar(50), Not Null	Full name of the doctor
PHC ID	Varchar(10), Not Null	Reference to the assigned PHC
Aadhaar Number	Varchar(12), Unique, Not Null	Aadhaar number of the doctor
Contact Number	Varchar(15), Not Null	Phone number of the doctor
Email	Varchar(100), Unique	Email of the doctor

3. HMO Table

Field Name	Data Type (Size, Constraints)	Description
HMO ID	Varchar(10), Primary Key	Unique ID of the HMO
Name	Varchar(50), Not Null	Full name of the HMO
Contact Number	Varchar(15), Not Null	HMO phone number
Email	Varchar(100), Unique	Email of the HMO
Region	Varchar(100), Not Null	Area or region supervised by HMO

4. DDHS Table

Field Name	Data Type (Size, Constraints)	Description
DDHS ID	Varchar(10), Primary Key	Unique ID of the DDHS
Name	Varchar(50), Not Null	Full name of the DDHS
Email	Varchar(100), Unique	Email of the DDHS
Contact Number	Varchar(15), Not Null	Phone number
District	Varchar(100), Not Null	District of supervision

5. PHC Table

Field Name	Data Type (Size, Constraints)	Description
PHC ID	Varchar(10), Primary Key	Unique ID for the PHC
PHC Name	Varchar(100), Not Null	Name of the PHC
Location	Varchar(150), Not Null	Physical location of the PHC
Contact Number	Varchar(15), Not Null	PHC contact number
Email	Varchar(100), Unique	PHC official email

6. PHC GeoFence Table

Field Name	Data Type (Size, Constraints)	Description
GeoFence ID	Varchar(10), Primary Key	Unique ID for the geofence
PHC ID	Varchar(10), Not Null	Reference to the PHC
Latitude	Varchar(50), Not Null	Latitude of geofence boundary
Longitude	Varchar(50), Not Null	Longitude of geofence boundary
Radius (Meters)	Int, Not Null	Radius of the geofence in meters
Created Date	Date, Not Null	Date the geofence was registered

7. Face Enrolment Table

Field Name	Data Type (Size, Constraints)	Description
Enrolment ID	Varchar(10), Primary Key	Unique enrolment ID
Doctor ID	Varchar(10), Not Null	Reference to the doctor
Face Image Path	Varchar(255), Not Null	File path to stored image
Enrolment Date	Date, Not Null	Date of face enrolment

8. Face Verification Table

Field Name	Data Type (Size, Constraints)	Description
Verification ID	Varchar(10), Primary Key	Unique ID for each verification
Doctor ID	Varchar(10), Not Null	Reference to the doctor
Photo Path	Varchar(255), Not Null	Path to the submitted photo
Latitude	Varchar(50), Not Null	GPS latitude value
Longitude	Varchar(50), Not Null	GPS longitude value
Verification Date	DateTime, Not Null	Date and time of verification
Verification Status	Varchar(20), Not Null	Status (e.g., Verified, Failed)

9. Attendance Table

Field Name	Data Type (Size, Constraints)	Description
Attendance ID	Varchar(10), Primary Key	Unique ID for each attendance entry
Doctor ID	Varchar(10), Not Null	Doctor who marked attendance
PHC ID	Varchar(10), Not Null	Reference to PHC
Date	Date, Not Null	Date of attendance
Time	Time, Not Null	Time of attendance
Status	Varchar(10), Not Null	Status (e.g., Present, Absent)

10. Patient Table

Field Name	Data Type (Size, Constraints)	Description
Patient ID	Varchar(10), Primary Key	Unique patient ID
Doctor ID	Varchar(10), Not Null	Doctor who attended the patient
Aadhaar Number	Varchar(12), Not Null	Patient Aadhaar number
Disease Type	Varchar(100), Not Null	Diagnosed disease
Medicine Given	Varchar(150), Not Null	Prescribed medicine
Visit Date	Date, Not Null	Date of patient visit

11. Alerts Table

Field Name	Data Type (Size, Constraints)	Description
Alert ID	Varchar(10), Primary Key	Unique ID for alert
Doctor ID	Varchar(10), Not Null	Target doctor of the alert
Alert Type	Varchar(50), Not Null	Type (SMS, Email, App)
Alert Message	Varchar(255), Not Null	Content of the alert
Alert Date	DateTime, Not Null	Date and time of alert sent

12. Escalation Table

Field Name	Data Type (Size, Constraints)	Description
Escalation ID	Varchar(10), Primary Key	Unique ID for escalation
Doctor ID	Varchar(10), Not Null	Reference to absentee doctor
PHC ID	Varchar(10), Not Null	Reference to PHC
Absenteeism Count	Int, Not Null	Number of absent days triggering alert
Action Taken	Varchar(100), Not Null	Action initiated (e.g., warning)
Escalated To	Varchar(50), Not Null	Who was notified (e.g., DDHS)
Escalation Date	Date, Not Null	Date of escalation

13. Reports Table

Field Name	Data Type (Size, Constraints)	Description
Report ID	Varchar(10), Primary Key	Unique ID for report
PHC ID	Varchar(10), Not Null	PHC reference
Report Type	Varchar(50), Not Null	Type (e.g., Monthly, Irregularity)
Generated By	Varchar(50), Not Null	Role or person who generated report
Generated Date	Date, Not Null	Date the report was created
Report File Path	Varchar(255), Not Null	Location of saved report file

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1. PROJECT DESCRIPTION

The proposed system is an AI-enabled, comprehensive platform designed to automate and streamline doctor attendance and monitoring at Primary Health Centers (PHCs). It addresses the limitations of traditional attendance mechanisms such as manual registers and basic biometric systems, which are prone to errors, manipulation, and lack real-time verification. The system integrates Geotagged photo uploads with CNN-based facial recognition to authenticate doctors' identities and ensure they are physically present within the PHC premises. By implementing polygon-based Geofencing and continuous GPS tracking, the platform monitors doctors throughout their duty hours, while random re-verification ensures sustained compliance. The system serves multiple end users, including PHC administrators, doctors, DDHS, and Health Ministry officials. Administrators can manage PHC records, define Geofence boundaries, register doctors, enroll facial data, monitor attendance, review violations, and generate reports. Doctors use the system to mark attendance, respond to random verification checks, and log patient consultations, linking workforce presence directly to healthcare service delivery. Supervisory authorities such as DDHS can monitor multiple PHCs, analyze attendance statistics, patient flow, and compliance trends, while Health Ministry officials gain insights at the regional and district level for policy-making and strategic planning. The platform's modules encompass Doctor Management, PHC Management, GeoFence establishment, Attendance Tracking, Violation Detection, Absenteeism Alerts, Escalation & Compliance, Patient Management, and Reporting & Analytics. Each module contributes to a secure, accountable, and efficient healthcare system, with real-time alerts, structured attendance logs, automated escalation of violations, and detailed performance analytics. By combining AI-based verification, geolocation tracking, automated alerts, and structured patient management, the system enhances governance, ensures workforce accountability, improves service delivery, and ultimately elevates the quality of healthcare provided to rural and semi-urban populations.

5.2. MODULE DESCRIPTION

1. PHC Doctor Attendance Dashboard

The design and development of the PHC Doctor Attendance Dashboard follow a user-centric approach to streamline attendance management and healthcare monitoring. The Front-End is built using Bootstrap, ensuring a responsive and intuitive interface that adapts to desktops, tablets, and smartphones. Python with Flask powers the Back-End, providing secure functionalities such as doctor authentication, attendance logic,

and role-based access control for administrators, doctors, and health officials. MySQL serves as the database to store attendance records, geolocation data, patient logs, and alert history. Key features include CNN-based facial recognition for identity verification, geofence-based location validation, continuous monitoring, random re-verification, violation alerts, patient consultation logging, and automated reporting. The system facilitates seamless monitoring, accountability, and decision support for PHCs, improving healthcare service delivery and administrative efficiency.

2. End User

2.1 PHC Administrators

PHC Administrators manage and oversee the daily operations of the system at individual health centers.

- **Login and Authentication:** Administrators securely access the dashboard using authorized credentials.
- **PHC Management:** Admins create and update PHC records, define geofence boundaries, and manage PHC details.
- **Doctor Management:** Admins register doctors, enroll facial data, and maintain their attendance records.
- **User Account Management:** Admins add and manage DDHS and Health Ministry accounts.
- **Attendance Monitoring:** Track doctor attendance logs, review violations, and ensure compliance.
- **Report Generation:** Generate automated reports for administrators, block-level authorities, and health officials.
- **Alerts & Notifications:** Receive alerts for absenteeism, geofence violations, and other compliance issues.

2.2 Doctors

Doctors primarily interact with the system to mark attendance and log patient consultations.

- **Login and Authentication:** Doctors securely log in to the mobile app with assigned credentials.
- **Attendance Capture:** Upload geo-tagged facial images to verify presence at PHC.
- **Random Re-Verification:** Respond to periodic random face verification requests during duty hours.
- **Alert Handling:** Receive notifications for missed check-ins, violations, or absenteeism.
- **Patient Activity Logging:** Record daily patient consultations including disease type and medicines prescribed.
- **Data Accuracy:** Maintain correct attendance and patient service records to support monitoring.

2.3 DDHS (Deputy Director of Health Services)

DDHS monitors PHC operations within their jurisdiction to ensure compliance and service quality.

- **Login and Authentication:** Secure access to the system with authorized credentials.
- **PHC Monitoring:** View consolidated attendance statistics and geolocation compliance across PHCs.
- **Patient Flow Analysis:** Track patient consultations and service delivery at multiple centers.
- **Violation Oversight:** Review absenteeism, geofence breaches, and attendance irregularities.
- **Corrective Actions:** Initiate notifications, warnings, or disciplinary measures as needed.
- **Report Generation:** Produce compliance and performance reports for administrators and higher authorities.

2.4 Health Ministry Officials

Health Ministry Officials access high-level dashboards for policy and governance purposes.

- **Login and Authentication:** Secure access with authorized credentials.
- **Regional Monitoring:** View district or state-level attendance trends and workforce utilization.
- **Analytics & Reporting:** Evaluate absenteeism patterns, patient volumes, and PHC performance metrics.
- **Policy Support:** Use data insights to guide strategic planning, staffing, and healthcare policies.
- **Alerts & Notifications:** Receive system alerts for critical compliance issues or persistent absenteeism.

3. PHC Management

The PHC Management module is designed to efficiently handle all operational aspects of Primary Health Centers within the attendance monitoring system. It allows administrators to create new health centers, update location details, assign doctors, link PHCs to supervisors, and manage the operational status of each center. By maintaining accurate records of doctor assignments, PHC locations, and administrative oversight, this module ensures proper coordination and accountability. Administrators can enable or disable centers based on operational requirements, while centralized monitoring enables seamless supervision and decision-making, ensuring reliable healthcare delivery and compliance at the local level.

3.1 GeoFence PHCs

The GeoFence PHCs module establishes secure virtual boundaries around each Primary Health Center using a polygon-based geofencing approach. Administrators define operational zones by marking multiple GPS coordinates on a digital map, which are stored in the database for real-time reference. When doctors mark attendance, their GPS location is validated against these geofences to ensure they are physically present within authorized areas. Any location outside the defined boundary triggers alerts for violations, absenteeism, or non-compliance. This module forms the foundation for location-based verification, random interval face re-verification, and continuous duty-hour monitoring, strengthening accountability and operational efficiency in PHCs.

4. Doctors Management

The Doctors Management module is responsible for maintaining comprehensive and structured records of all medical officers registered in the system. It ensures accurate mapping between doctors and their assigned PHCs, enabling traceability, accountability, and seamless integration with attendance and patient monitoring functionalities.

4.1 Doctor Registration

The Doctor Registration submodule captures essential professional and identity information of doctors, including full name, designation, registration number, assigned PHC, contact details, and duty schedule. It generates secure login credentials and activates doctor accounts, ensuring that each doctor is properly registered in the system. This structured registration process guarantees that all doctors are traceable, accountable, and correctly linked to their respective health centers for efficient monitoring.

4.2 Doctor Face Enrollment

The Doctor Face Enrollment submodule is designed to prepare high-quality biometric data for AI-based identity verification.

4.2.1 Dataset Creation

This process involves capturing a live video of the doctor's face for approximately 30 seconds using the system camera. Multiple facial angles, head movements, and expressions are recorded to build a diverse dataset, enhancing recognition accuracy under varying lighting conditions and facial variations.

4.2.2 Frame Conversion

The captured video is automatically converted into individual image frames. These frames serve as structured input samples for preprocessing and model training, enabling efficient handling of facial data.

4.2.3 Image Preprocessing

Each frame undergoes preprocessing to improve image quality and reduce noise. Techniques include resizing, grayscale conversion, median filtering, binarization, and foreground–background separation. This ensures consistent input quality for the AI model and enhances feature extraction.

4.2.4 Face Detection

The system applies the MTCNN (Multi-task Cascaded Convolutional Neural Network) algorithm to detect and crop facial regions accurately. This step isolates the face from the background, removing interference and enabling precise facial analysis.

4.2.5 Feature Extraction

Facial features are extracted using convolutional neural network layers with convolution layers, ReLU activation functions, and pooling layers. These operations transform raw image data into meaningful numerical representations of unique facial characteristics.

4.2.6 Classification

Extracted features are processed through flatten layers, fully connected (dense) layers, and a softmax output layer. This classification organizes identity-specific features and prepares the data for model training.

4.2.7 FaceNet Model Training

The processed facial data is used to train the FaceNet model, generating unique facial embeddings for each doctor. These embeddings act as digital identity signatures for real-time comparisons during attendance verification.

4.2.8 Model Deployment

The trained FaceNet model is integrated into the PHC Doctor Attendance Dashboard and mobile system. It enables real-time identity verification for marking attendance and supports random geo-tagged face re-verification, ensuring continuous accountability and compliance throughout duty hours.

5. Doctor Attendance Tracking

The Doctor Attendance Tracking module manages the complete lifecycle of attendance recording, validation, and monitoring for doctors at Primary Health Centers. It ensures accurate identity verification, geolocation compliance, continuous duty-hour tracking, and maintains comprehensive attendance logs for administrative oversight.

5.1 Geo-Tagged Photo Upload

This submodule enables doctors to upload a live selfie through the mobile or web application with geotagging enabled. The captured image contains embedded GPS coordinates, which are extracted and

stored along with a timestamp. By combining identity data with physical location, this module ensures that attendance is verified in real time and tied to the authorized PHC premises.

5.2 Face Biometric Recognition

The uploaded image is processed using a CNN-based facial recognition system integrated with the trained FaceNet model. The system compares the captured face against the doctor's enrolled facial dataset to confirm identity, preventing proxy attendance. This step ensures high-accuracy authentication before marking attendance as valid, securing the system against unauthorized access.

5.3 Geo-Location Validation

This submodule validates whether the doctor's GPS coordinates fall within the predefined PHC geofence using the Ray Casting algorithm. It checks whether the location lies inside the polygon-based secure boundary and applies time-based validation to ensure attendance is submitted within designated shift hours. Any deviation from location or timing rules marks the attendance as invalid, reinforcing accountability.

5.4 Continuous Monitoring

During duty hours, the system periodically collects GPS updates at fixed intervals, typically every 10–15 minutes, to ensure the doctor remains within PHC premises. Any prolonged absence, GPS inactivity, or unusual movement is flagged for review. This continuous tracking ensures compliance throughout the duty shift and supports real-time enforcement.

5.5 Random Re-Verification

To prevent temporary check-ins or proxy attendance, the system triggers random re-verification requests during duty hours. Doctors must submit a new live facial image with GPS data, which is validated again using the CNN model and geofencing logic. Non-compliance or failed verification is flagged as a potential violation, reinforcing continuous presence.

5.6 Attendance Management

This submodule maintains structured attendance logs, including check-in times, total presence duration, verification results, and compliance status. It supports audit trails, historical attendance tracking, and report generation for PHC administrators and higher health authorities. The module provides actionable insights to monitor workforce accountability and improve healthcare service delivery.

6. Violation Detection

The Violation Detection module continuously monitors and analyzes attendance-related data, including identity verification results, GPS location logs, check-in/out timings, and overall system activity. It identifies irregularities such as unauthorized exits, failed facial recognition attempts, missed random re-

verifications, delayed check-ins, and prolonged GPS inactivity. Detected violations are automatically logged and prioritized based on severity, then forwarded to the alert and escalation modules for immediate action.

This proactive monitoring ensures that workforce compliance is maintained and reduces risks associated with absenteeism, ultimately improving healthcare service reliability.

7. Absenteeism Alerts

This module generates instant notifications whenever attendance anomalies or violations are detected. Alerts are sent to doctors, PHC administrators, and supervisory authorities via in-app messages, emails, or SMS. By informing the concerned parties in real time, the system allows corrective measures to be taken promptly, preventing prolonged disruptions in patient care. The module also tracks recurring absenteeism patterns to help administrators identify chronic non-compliance.

8. Escalation & Compliance

The Escalation & Compliance module handles repeated or critical violations by escalating detailed reports to higher authorities such as the DDHS or Health Ministry officials. It maintains a structured record of disciplinary actions, historical violations, and notifications sent, ensuring transparency and accountability. This module helps enforce rules consistently across all PHCs, strengthens adherence to duty schedules, and provides data-driven support for policy enforcement and corrective action decisions.

9. Patient Management

The Patient Management module allows doctors to digitally record all patient interactions, including consultations, diagnoses, treatments, and prescribed medications. Each entry is timestamped and correlated with the doctor's attendance records, linking service delivery with actual presence at the PHC. This enables administrators to monitor productivity, identify gaps in healthcare coverage, and maintain accurate patient histories. The module also supports reporting for health outcomes, enabling better planning and allocation of resources in rural healthcare settings.

10. Reporting & Analytics

The Reporting & Analytics module generates comprehensive and visual reports for operational, administrative, and policy-level use. It provides daily attendance summaries, monthly compliance trends, geolocation movement logs, violation statistics, and patient flow analysis. Reports can be exported in

standard formats, and the analytics dashboard offers charts and visualizations for quick insights. By combining attendance, compliance, and patient data, this module supports data-driven decision-making, identifies patterns of absenteeism, and helps optimize workforce allocation, thereby improving the efficiency and quality of healthcare delivery in rural areas.

5.3. IMPLEMENTATION

Requirement Setup: This stage identifies all system requirements such as doctor login, face recognition attendance, GPS verification, geofencing, continuous location tracking, alerts, and patient data entry. Security, accuracy, and usability requirements are also finalized.

Database Creation: A secure database is designed to store doctor details, attendance records, GPS logs, alerts, and patient consultation data. Proper table structure ensures easy data management and real-time monitoring.

Frontend Development: A user-friendly mobile app is developed for doctors to mark attendance, while an admin dashboard is created for monitoring attendance, location status, and alerts. The interface is designed to be simple and responsive.

Backend Development: The backend manages authentication, attendance processing, GPS validation, alerts, and database communication. It acts as the main connection between the frontend, AI model, and storage system.

Algorithm Integration: A CNN-based facial recognition model is integrated to verify doctor identity. Along with this, GPS-based geofencing checks whether the doctor is present within the PHC boundary, ensuring genuine attendance.

System Integration: All modules including frontend, backend, AI model, database, and GPS tracking are combined into one complete system. This enables real-time attendance and continuous duty-hour monitoring.

Testing and Validation: The system is tested for face recognition accuracy, GPS precision, and overall performance. Functional and integration testing help identify and fix errors before deployment.

User Training and Documentation: Doctors and PHC staff are trained to use the system effectively. User manuals and technical documentation are pr

CHAPTER 6

SYSTEM TESTING

6.1. SOFTWARE TESTING

Software testing is a critical phase in the development of the project, ensuring that the system operates as intended and meets the required standards. Testing helps to identify and address defects early, verifying the functionality, performance, security, and usability of the system. By conducting comprehensive testing, we aim to improve the reliability and efficiency of the system while minimizing potential errors.

6.1.1. Types of Testing

Unit Testing: Unit testing focuses on individual components or functions of the system. Each module, such as doctor login, face recognition, geolocation verification, attendance logging, violation detection, and patient data recording, is tested separately to ensure all features work as expected. For example, the system verifies that the FaceNet model correctly matches a doctor's face or that geofence validation accurately confirms the doctor's location.

Integration Testing: Integration testing ensures that various modules of the system function together seamlessly. This includes verifying that doctor login integrates correctly with attendance marking, facial verification, and geolocation tracking. The goal is to ensure data flows correctly between modules and the system operates as a cohesive unit.

System Testing: System testing evaluates the entire system as a whole. This phase tests the system in a real-world environment to ensure all features, including continuous attendance monitoring, alerts, escalations, and reporting, function as intended. System testing also checks non-functional requirements such as performance, scalability, and reliability.

User Acceptance Testing (UAT): User acceptance testing involves end-users, including PHC administrators, doctors, DDHS officials, and health ministry authorities, testing the system from their perspective. The purpose of UAT is to validate that the system meets user expectations, is intuitive, and allows seamless attendance marking, monitoring, and report generation. It confirms that the features align with the initial project objectives.

Performance Testing: Performance testing assesses how the system handles different workloads and scenarios. This includes testing response times during simultaneous doctor logins, bulk attendance submissions, or high-frequency location tracking. Performance testing ensures the system remains responsive, stable, and efficient under various load conditions.

Security Testing: Security testing focuses on protecting sensitive data, such as doctor credentials, GPS locations, and patient information. It verifies encryption of sensitive data, prevents unauthorized access, and protects against threats like SQL injection or cross-site scripting (XSS). Authentication and authorization mechanisms are also tested to ensure only valid users access the system.

Usability Testing: Usability testing evaluates how user-friendly the system is for doctors, administrators, and supervisory authorities. It ensures all users can navigate the platform easily, mark attendance, respond to re-verifications, view alerts, and access reports. Feedback from users is collected to improve the interface and overall experience.

Regression Testing: Regression testing is performed after updates, enhancements, or bug fixes to ensure existing functionalities are not disrupted. Features such as attendance validation, violation alerts, GPS tracking, and report generation are retested to maintain system stability and reliability after each change.

6.2. TEST CASES

Test Case ID: TC001

- **Input:** Doctor logs in using valid username and password.
- **Expected Result:** System successfully authenticates the doctor and opens the attendance dashboard.
- **Actual Result:** Doctor is logged in and redirected to the dashboard.
- **Status:** Pass

Test Case ID: TC002

- **Input:** Doctor attempts to log in with invalid credentials.
- **Expected Result:** System displays an error message indicating invalid login details.
- **Actual Result:** Error message is displayed and login is denied.
- **Status:** Pass

Test Case ID: TC003

- **Input:** Doctor uploads a live facial image for attendance marking.
- **Expected Result:** System captures the image and sends it for facial verification.
- **Actual Result:** Image is successfully captured and processed.
- **Status:** Pass

Test Case ID: TC004

- **Input:** Uploaded facial image matches registered facial data.
- **Expected Result:** Attendance is marked successfully.
- **Actual Result:** Attendance is recorded with timestamp and location.

- **Status:** Pass

Test Case ID: TC005

- **Input:** Uploaded facial image does not match registered face data.
- **Expected Result:** Attendance is rejected with a verification error.
- **Actual Result:** Attendance is rejected and alert is generated.
- **Status:** Pass

Test Case ID: TC006

- **Input:** Doctor marks attendance outside PHC geofence.
- **Expected Result:** System rejects attendance due to invalid location.
- **Actual Result:** Attendance is denied and location violation is logged.
- **Status:** Pass

Test Case ID: TC007

- **Input:** Doctor remains outside PHC boundary during duty hours.
- **Expected Result:** System detects geo-violation and generates alert.
- **Actual Result:** Alert is triggered and sent to administrator.
- **Status:** Pass

Test Case ID: TC008

- **Input:** Doctor fails to respond to random re-verification request.
- **Expected Result:** System flags absence and logs violation.
- **Actual Result:** Violation recorded and alert sent.
- **Status:** Pass

Test Case ID: TC009

- **Input:** Doctor submits attendance within scheduled duty time.
- **Expected Result:** Attendance is marked as valid.
- **Actual Result:** Attendance successfully recorded.
- **Status:** Pass

Test Case ID: TC010

- **Input:** Doctor attempts attendance after allowed time window.
- **Expected Result:** Attendance is rejected as late check-in.
- **Actual Result:** Late entry rejected and logged.
- **Status:** Pass

Test Case ID: TC011

- **Input:** Administrator views attendance report.
- **Expected Result:** System displays complete attendance details.
- **Actual Result:** Attendance report generated correctly.
- **Status:** Pass

Test Case ID: TC012

- **Input:** Administrator views violation records.
- **Expected Result:** All violations with timestamps are displayed.
- **Actual Result:** Violation report generated successfully.
- **Status:** Pass

Test Case ID: TC013

- **Input:** Doctor enters patient consultation details.
- **Expected Result:** Patient data is saved with timestamp.
- **Actual Result:** Patient records stored successfully.
- **Status:** Pass

Test Case ID: TC014

- **Input:** DDHS views PHC-wise attendance summary.
- **Expected Result:** System displays consolidated attendance report.
- **Actual Result:** Summary report generated accurately.
- **Status:** Pass

Test Case ID: TC015

- **Input:** Health Ministry official views analytics dashboard.

- **Expected Result:** System displays charts and trends of attendance and compliance.
- **Actual Result:** Dashboard loads correctly with analytics data.
- **Status:** Pass

6.3. TEST REPORT

Introduction

The project underwent comprehensive testing to ensure its reliability, accuracy, and effectiveness in tracking doctor attendance using AI-based facial recognition and geolocation verification. The purpose of testing was to validate that all system modules function correctly, securely, and efficiently under real-time conditions. This report documents the testing objectives, scope, environment, results, and identified issues during system validation.

Test Scope

The testing process covered the following functional and non-functional components:

- User authentication and role-based login
- Doctor face enrollment and recognition
- Geo-tagged attendance validation
- Geofence boundary verification
- Continuous GPS tracking
- Random re-verification process
- Violation detection and alert generation
- Patient management module
- Report generation and analytics dashboard

Test Environment

- **Hardware:** Intel Core i5 / i7, 8 GB RAM, 500 GB SSD
- **Operating System:** Windows 10/11
- **Backend:** Python, Flask
- **Frontend:** HTML, CSS, Bootstrap
- **Database:** MySQL
- **AI Frameworks:** TensorFlow, OpenCV
- **Testing Tools:** Selenium, Postman, JMeter
- **Browsers:** Google Chrome, Mozilla Firefox, Microsoft Edge

Test Results

Most of the test cases were executed successfully and produced expected outcomes. The system accurately verified doctor identity using facial recognition, validated location using geofencing, and generated alerts for violations. Attendance tracking, reporting, and patient management modules worked as intended. Minor issues were identified related to GPS accuracy and notification delays, which were logged for correction.

Bug Report

Bug ID	TC ID	Bug Description	Bug Status	Output
B001	TC006	Attendance marked without GPS data in rare cases	Fixed	GPS validation improved
B002	TC008	Face recognition failed under low lighting	Fixed	Model retrained
B003	TC010	Delay in alert notification	Fixed	Alert timing optimized
B004	TC012	Report loading slow for large data	Fixed	Query optimization applied

Test Conclusion

The project successfully passed the majority of functional and performance tests. Core features such as facial authentication, geolocation verification, attendance tracking, violation detection, and reporting worked efficiently. Minor issues identified during testing were resolved through optimization and validation improvements. Overall, the system meets the required objectives and is reliable for deployment in real-world Primary Health Center environments. The project effectively enhances accountability, transparency, and operational efficiency in rural healthcare management.

CHAPTER 7

CONCLUSION

7.1. CONCLUSION

In conclusion, the comprehensive design and development of the project is centred on improving transparency, accountability, and efficiency in healthcare service delivery. Built using Python with Flask for backend processing, Bootstrap for a responsive user interface, and MySQL for structured data management, the system ensures reliable and secure handling of attendance and patient information. Key features such as face recognition–based attendance, geofenced location verification, real-time monitoring, automated alerts, and escalation mechanisms effectively eliminate proxy attendance and improve compliance among doctors. The integration of patient management, violation tracking, and analytical reporting further strengthens administrative control and supports informed decision-making. Modules such as doctor registration, attendance tracking, geolocation validation, violation detection, and reporting work together to streamline operations and enhance accountability at all levels of healthcare administration. Overall, the PHC Doctor Attendance System offers a robust, technology-driven solution that modernizes traditional attendance practices, improves workforce management, and contributes to better healthcare governance and service delivery.

7.2. FUTURE ENHANCEMENT

The project can be further enhanced to improve accuracy, scalability, and overall effectiveness in healthcare administration. Several advanced features can be integrated in future versions to strengthen system performance and expand its applicability.

Advanced Anti-Spoofing and Liveness Detection

Future upgrades can incorporate advanced biometric security mechanisms such as eye-blink detection, 3D facial depth analysis, and infrared-based face scanning. These techniques will help prevent photo, video, or mask-based spoofing attacks and significantly enhance the reliability of facial authentication.

Expansion to Other Public Services

The system can be extended beyond doctors to include nurses, field health workers, ambulance drivers, and other government healthcare staff. This expansion will enable centralized monitoring of workforce attendance across multiple departments, improving operational efficiency and accountability in public service delivery.

Cloud Deployment and Mobile Optimization

Migrating the system to a cloud-based infrastructure can improve scalability, data availability, and disaster recovery. Additionally, enhanced mobile application support can ensure seamless access for doctors and administrators in remote areas.

APPENDIX

SAMPLE SOURCE CODE

Packages

```
from flask import Flask, render_template, Response, redirect, request, session, abort, url_for
from flask_mail import Mail, Message
from camera import VideoCamera
from flask import send_file
import cv2
import numpy as np
import pandas as pd
import shutil
import hashlib
from random import randint
from PIL import Image
import imagehash
import geocoder
from datetime import datetime
from PIL import Image, ImageDraw, ImageFont
import mysql.connector
from werkzeug.utils import secure_filename
from shapely.geometry import Point, Polygon
```

Database Connection

```
mydb = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    charset="utf8",
    database="phc_doctor"
)
```

Login

```
def login():
    msg=""
    if request.method=='POST':
        uname=request.form['uname']
        pwd=request.form['pass']
        cursor = mydb.cursor()
        cursor.execute('SELECT * FROM ph_admin WHERE username = %s AND password = %s',
(uname, pwd))
        account = cursor.fetchone()
        if account:
            session['username'] = uname
            # Redirect to home page
            return redirect(url_for('admin'))
        else:
            msg = 'Incorrect username/password!'
```

Add Doctor

```
def add_doctor():
    msg=""
    mess=""
    email=""
    vid=""
    act=request.args.get("act")
    data=[]
    st=""
    mycursor = mydb.cursor()

    mycursor.execute("SELECT max(id)+1 FROM ph_doctor")
    maxid = mycursor.fetchone()[0]
```

```

if maxid is None:
    maxid=1
docid="D"+str(maxid)
mycursor.execute("SELECT * FROM ph_hospital")
phos = mycursor.fetchall()
if request.method=='POST':
    name=request.form['name']
    gender=request.form['gender']
    mobile=request.form['mobile']
    email=request.form['email']
    district=request.form['district']
    specialized=request.form['specialized']
    uname=request.form['uname']
    pass1=request.form['pass']
    phc_id=request.form['phc_id']
    stime=request.form['stime']
    etime=request.form['etime']
    smin=request.form['smin']
    emin=request.form['emin']
    mycursor.execute("SELECT count(*) FROM ph_doctor where uname=%s",(uname,))
    cnt = mycursor.fetchone()[0]
    if cnt==0:
        sql = "INSERT INTO
ph_doctor(id,name,gender,mobile,email,district,specialized,uname,pass,phc_id,stime,etime,smin,emin)
VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)"
        val =
(maxid,name,gender,mobile,email,district,specialized,uname,pass1,phc_id,stime,etime,smin,emin)
        mycursor.execute(sql, val)
        mydb.commit()
        vid=str(maxid)
        mess="Dear "+name+", Username: "+uname+", Password: "+pass1

```

```
msg="success"
```

Training – Face

```
#Preprocess
```

```
path="static/frame/"+rs[2]
path2="static/process1/"+rs[2]
mm2 = PIL.Image.open(path).convert('L')
rz = mm2.resize((200,200), PIL.Image.ANTIALIAS)
rz.save(path2)
img = cv2.imread(path2)
dst = cv2.fastNlMeansDenoisingColored(img, None, 10, 10, 7, 15)
path3="static/process2/"+rs[2]
cv2.imwrite(path3, dst)
#noice
img = cv2.imread('static/process1/'+rs[2])
dst = cv2.fastNlMeansDenoisingColored(img, None, 10, 10, 7, 15)
fname2='ns_'+rs[2]
#binariazation
image = cv2.imread('static/process1/'+rs[2])
original = image.copy()
kmeans = kmeans_color_quantization(image, clusters=4)
# Convert to grayscale, Gaussian blur, adaptive threshold
gray = cv2.cvtColor(kmeans, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (3,3), 0)
thresh = cv2.adaptiveThreshold(blur,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY_INV,21,2)
# Draw largest enclosing circle onto a mask
mask = np.zeros(original.shape[:2], dtype=np.uint8)
cnts = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cnts = cnts[0] if len(cnts) == 2 else cnts[1]
```

```

cnts = sorted(cnts, key=cv2.contourArea, reverse=True)
for c in cnts:
    ((x, y), r) = cv2.minEnclosingCircle(c)
    cv2.circle(image, (int(x), int(y)), int(r), (36, 255, 12), 2)
    cv2.circle(mask, (int(x), int(y)), int(r), 255, -1)
    break
# Bitwise-and for result
result = cv2.bitwise_and(original, original, mask=mask)
result[mask==0] = (0,0,0)
cv2.imshow('image', image)
cv2.imwrite("static/process1/bin_"+rs[2], thresh)
#RPN - Segment
img = cv2.imread('static/process1/'+rs[2])
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
ret, thresh = cv2.threshold(gray,0,255,cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
kernel = np.ones((3,3),np.uint8)
opening = cv2.morphologyEx(thresh,cv2.MORPH_OPEN,kernel, iterations = 2)
# sure background area
sure_bg = cv2.dilate(opening,kernel,iterations=3)
# Finding sure foreground area
dist_transform = cv2.distanceTransform(opening,cv2.DIST_L2,5)
ret, sure_fg = cv2.threshold(dist_transform,0.7*dist_transform.max(),255,0)
# Finding unknown region
sure_fg = np.uint8(sure_fg)
segment = cv2.subtract(sure_bg,sure_fg)
img = Image.fromarray(img)
segment = Image.fromarray(segment)
path3="static/process2/fg_"+rs[2]
segment.save(path3)

```

#Feature extraction & Classification

```

def CNN_process(self):
    train_data_preprocess = ImageDataGenerator(
        rescale = 1./255,
        shear_range = 0.2,
        zoom_range = 0.2,
        horizontal_flip = True)
    test_data_preprocess = (1./255)
    train = train_data_preprocess.flow_from_directory(
        'dataset/training',
        target_size = (128,128),
        batch_size = 32,
        class_mode = 'binary')
    test = train_data_preprocess.flow_from_directory(
        'dataset/test',
        target_size = (128,128),
        batch_size = 32,
        class_mode = 'binary')

    # Initialising the CNN
    # Step 1 - Convolution
    # Step 2 - Pooling
    cnn.add(Conv2D(32, (3, 3), input_shape = (128, 128, 3), activation = 'relu'))
    cnn.add(MaxPooling2D(pool_size = (2, 2)))
    # Adding a second convolutional layer
    cnn.add(Conv2D(32, (3, 3), activation = 'relu'))
    cnn.add(MaxPooling2D(pool_size = (2, 2)))
    # Step 3 - Flattening
    cnn.add(Flatten())
    # Step 4 - Full connection
    cnn.add(Dense(units = 128, activation = 'relu'))
    cnn.add(Dense(units = 1, activation = 'sigmoid'))

```

```

# Compiling the CNN
cnn.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['accuracy'])
history = cnn.fit_generator(train,
                            steps_per_epoch = 250,
                            epochs = 25,
                            validation_data = test,
                            validation_steps = 2000)

plt.plot(history.history['acc'])
plt.plot(history.history['val_acc'])
plt.title('Model Accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()

test_image = image.load_img('\\dataset\\', target_size=(128,128))
test_image = image.img_to_array(test_image)
test_image = np.expand_dims(test_image, axis=0)
result = cnn.predict(test_image)
print(result)
if result[0][0] == 1:
    print('feature extracted and classified')
else:

```



```
print('none')
```

Geo Tagged Attendance

```
if act=="capture":
```

```
    image_path = "static/getimg.jpg" # Change this to your image file
```

```
    image = cv2.imread(image_path)
```

```
    if image is not None:
```

```
        # Get GPS Coordinates and Current Time
```

```
        latitude, longitude = get_gps_coordinates()
```

```
        current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
        gps_text = f"Lat: {latitude:.6f}, Lon: {longitude:.6f}"
```

```
        rd=current_time.split(" ")
```

```
        cdate=rd[0]
```

```
        rd1=rd[1].split(":")
```

```
        chour=int(rd1[0])
```

```
        cmin=int(rd1[1])
```

```
        lt=gps_text.split(",")
```

```
        lt1=lt[0].split(":")
```

```
        lt2=lt[1].split(":")
```

```
        lat=lt1[1].strip()
```

```
        lon=lt2[1].strip()
```

```
        # Convert OpenCV image to PIL format
```

```
        img_pil = Image.fromarray(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
```

```
        draw = ImageDraw.Draw(img_pil)
```

```
        # Define Box Position (Bottom Left)
```

```
        img_w, img_h = img_pil.size
```

```

rect_w, rect_h = 350, 80
rect_x = 20 # Positioned at the left side
rect_y = img_h - rect_h - 20
radius = 20
rectangle = Image.new("RGB", (rect_w, rect_h), (255, 255, 255))
mask = Image.new("L", (rect_w, rect_h), 0)
draw_mask = ImageDraw.Draw(mask)
draw_mask.rounded_rectangle((0, 0, rect_w, rect_h), radius, fill=255)
# Paste Rectangle onto Image
img_pil.paste(rectangle, (rect_x, rect_y), mask)
# Load Font and Draw Text
font = ImageFont.truetype("static/fonts/ARIAL.ttf", 20)
text_x, text_y = rect_x + 20, rect_y + 10 # Offset from the box
time_x, time_y = rect_x + 20, rect_y + 40 # Position time below GPS

draw.text((text_x, text_y), gps_text, font=font, fill=(0, 0, 0))
draw.text((time_x, time_y), f"Time: {current_time}", font=font, fill=(0, 0, 0))
# Convert back to OpenCV format
final_img = cv2.cvtColor(np.array(img_pil), cv2.COLOR_RGB2BGR)
# Save and Show Image
output_path = "static/capture/"+dimg
cv2.imwrite(output_path, final_img)
print(f"Geo-tagged image saved as: {output_path}")
image_path = "static/capture/"+dimg
latitude, longitude, timestamp = get_gps_and_time(image_path)

#GPS Coordinates Validation [Ray Casting Algorithm]
mycursor.execute("SELECT * FROM ph_hospital where phc_id=%s",(phc_id,))
d4 = mycursor.fetchone()
g1=d4[5]

```

```

g2=g1.split('new google.maps.LatLng(')
g21="".join(g2)
g22=g21.split('), ')
print(g22)
glen=len(g22)
coors=[]
i=1
for g3 in g22:
    cr=[]
    if i<glen:
        g31=g3.split(",")
        print(g31)
        g32=float(g31[0])
        g33=float(g31[1])
        cr.append(g32)
        cr.append(g33)
        coors.append(cr)
    i+=1
latitude = float(lat)
longitude = float(lon)
point = Point(longitude, latitude) # Note: shapely uses (x, y) = (lon, lat)
polygon_coords = coors
polygon = Polygon(polygon_coords)

# Check if the point is inside the polygon
if polygon.contains(point):
    s=1
    print("The point is inside the polygon.")
else:
    s=2

```

```

    print("The point is outside the polygon.")
l1=lat1[1]
l2=lat1[0:4]
lt=int(l2)
gn=len(g2)-1
i=0
gloc1.sort()
#print(gloc1)
gn=len(gloc1)-1
gn1=int(gloc1[0])
gn2=int(gloc1[gn])
status=""
if lt==geo1 and lo==geo2:
    if gn1<=lt and lt<=gn2:
        status="1"
        print("geo")
    else:
        status="2"
        mess2="Incorrect Location"
#Time Based Validity
tstatus="2"
if stime<=chour and chour<=etime:
    gmin=smin+maxtime
    hour1=stime+1
    if gmin>=60:
        gmin1=gmin-60
        if stime==chour:
            if smin<=cmin and cmin<60:
                tstatus="1"
        elif hour1==chour:

```

```

        if cmin>=0 and cmin<=gmin1:
            tstatus="1"
        elif stime==chour:
            if smin<=cmin and cmin<=gmin:
                tstatus="1"
            else:
                tstatus="2"
                mess3="Incorrect Time"
        fstatus=""
        ff=open("static/cam1.txt","r")
        fst=ff.read()
        ff.close()
        if uname==fst:
            fstatus="1"
        else:
            fstatus="2"
            mess1="Face Not Matched"

        mycursor.execute("SELECT count(*) FROM ph_attendance where att_date=%s &&
docid=%s",(cdate,uname))

        cnt = mycursor.fetchone()[0]

        ""if cnt==0:

            sql = "INSERT INTO
ph_attendance(id,docid,phc_id,geo_image,latitude,longitude,att_date,att_time,face_st,geo_st,time_st)
VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)"

            val = (maxid,uname,phc_id,dimimg,lat,lon,rd[0],rd[1],fstatus,status,tstatus)

            mycursor.execute(sql, val)

            mydb.commit()

        else:""

        mycursor.execute("update ph_attendance set
geo_image=%s,latitude=%s,longitude=%s,att_time=%s,face_st=%s,geo_st=%s,time_st=%s where
att_date=%s && docid=%s",(dimimg,lat,lon,rd[1],fstatus,status,tstatus,cdate,uname))

        mydb.commit()

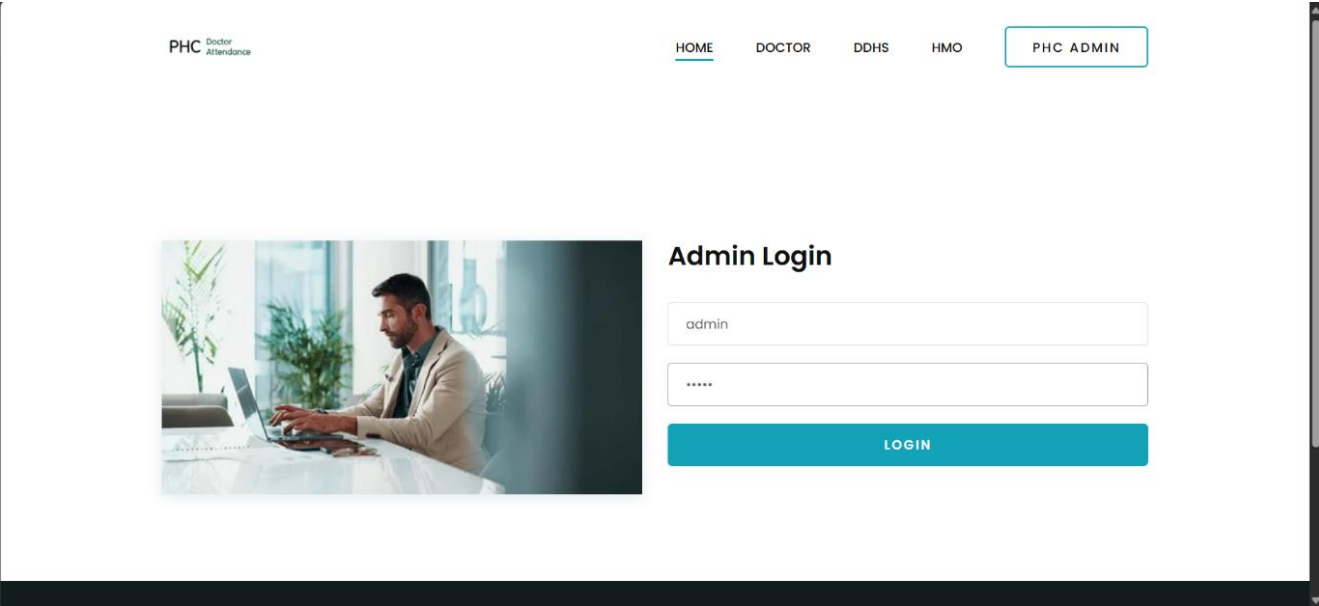
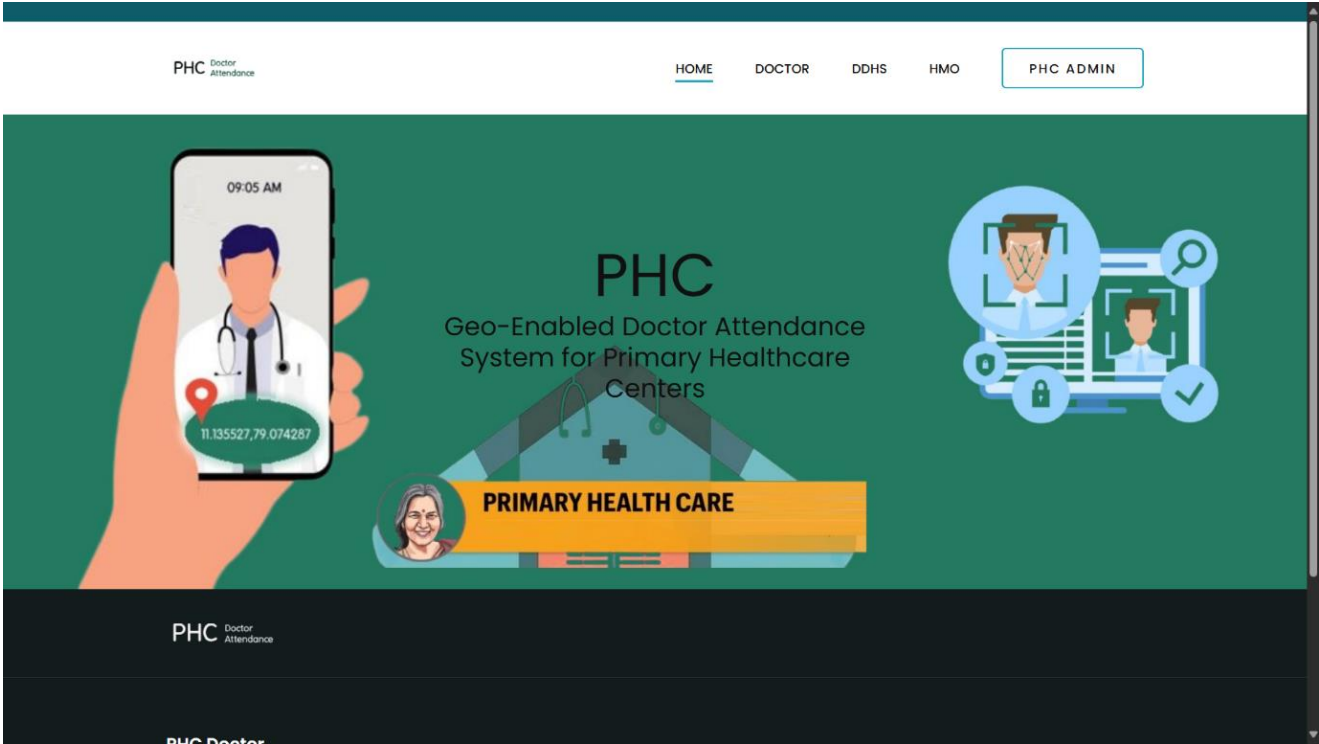
```

```

if fstatus=="2" and status=="2" and tstatus=="2":
    st="1"
    mess="Doctor ID: "+uname+", Incorrect Face,Time,Location"
elif fstatus=="2" and status=="2":
    st="1"
    mess="Doctor ID: "+uname+", Incorrect Face and Location"
elif fstatus=="2" and tstatus=="2":
    st="1"
    mess="Doctor ID: "+uname+", Incorrect Face and Time"
elif status=="2" and tstatus=="2":
    st="1"
    mess="Doctor ID: "+uname+", Incorrect Location and Time"
elif status=="2":
    st="1"
    mess="Doctor ID: "+uname+", Incorrect Location"
elif tstatus=="2":
    st="1"
    mess="Doctor ID: "+uname+", Incorrect Time"
elif fstatus=="2":
    st="1"
    mess="Doctor ID: "+uname+", Incorrect Face"
else:
    st="2"
    mess="Attendance Captured"

```

SAMPLE SCREEN SHOT



Admin Login

Admin

Dashboard

- Doctor
- DDHS
- HMO
- Logout

PHC

Add PHC Details

Add

PHC ID: P001

PHC Name	: GH1
Area	: RR
District	: Thanjavur

Geo Location / Delete

PHC Details

Logout

Delete Selected Shape

Copy to Clipboard

new

google.maps.LatLng(10.8

Type Area

RR

Add

Go Back

Location

Admin

Dashboard

Doctor

DDHS

HMO

Logout

Dashboard

Add Doctor Details

Name

Male

Mobile No.

Email

District

-Select-

Specialized in

Start Time

-Hour-

-Minute-

End Time

-Select-

-Minute-

Doctor ID: D2

Name

Gender

Mobile No.

Email

District

Specialized in

PHC ID

Time

: Dr. R.Geetha, M.D

: Female

: 8956222541

: geetha@gmail.com

: Chennai

: General Physician

: P001

: 9:0 to 17:0

Edit / Delete

Doctor ID: D1

Name

Gender

Mobile No.

Email

District

Specialized in

PHC ID

Time

: Dr. P.Raman, MBBS

: Male

: 9874522555

: raman@gmail.com

: Salem

: General Physician

: P001

: 10:0 to 16:30

Edit / Delete

Add Doctor Details

Dashboard

Doctor

DDHS

HMS

Logout

Dashboard

Deputy Director of Health Services

Add DDHS Details

Name

Mobile No.

Email

District

DH2

Password

Add

DDHS ID: DH1

Name

Mobile No.

Email

District

: Harish

: 8895624212

: harish@gmail.com

: Salem

Delete

Add DDHS Details

65

Admin

Dashboard

Doctor

DDHS

HMO

Logout

Health Ministry Officials

Add HMO Details

Name

Mobile No.

Email

District

HM2

Password

HMO ID: HM1

Name : Girish

Mobile No. : 7358542255

Email : girish@gmail.com

District : Salem

Delete

Add

Add HMO Details

PHC Doctor Attendance

HOME

DOCTOR

DDHS

HMO

PHC ADMIN

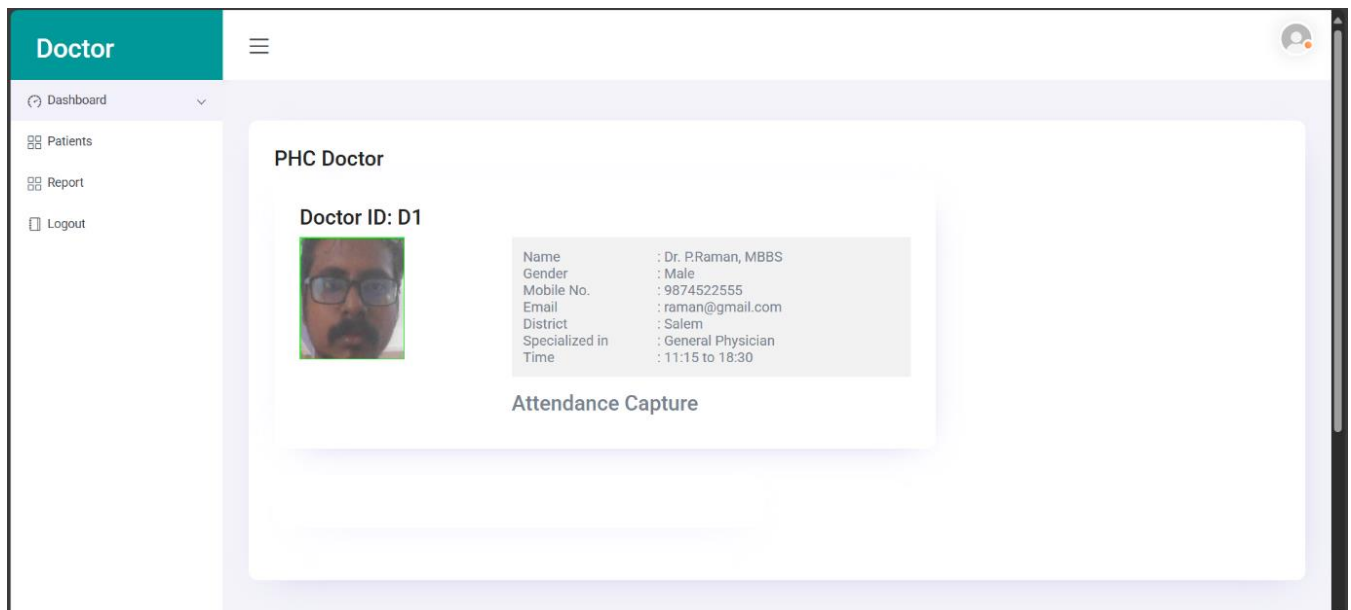


Doctor Login

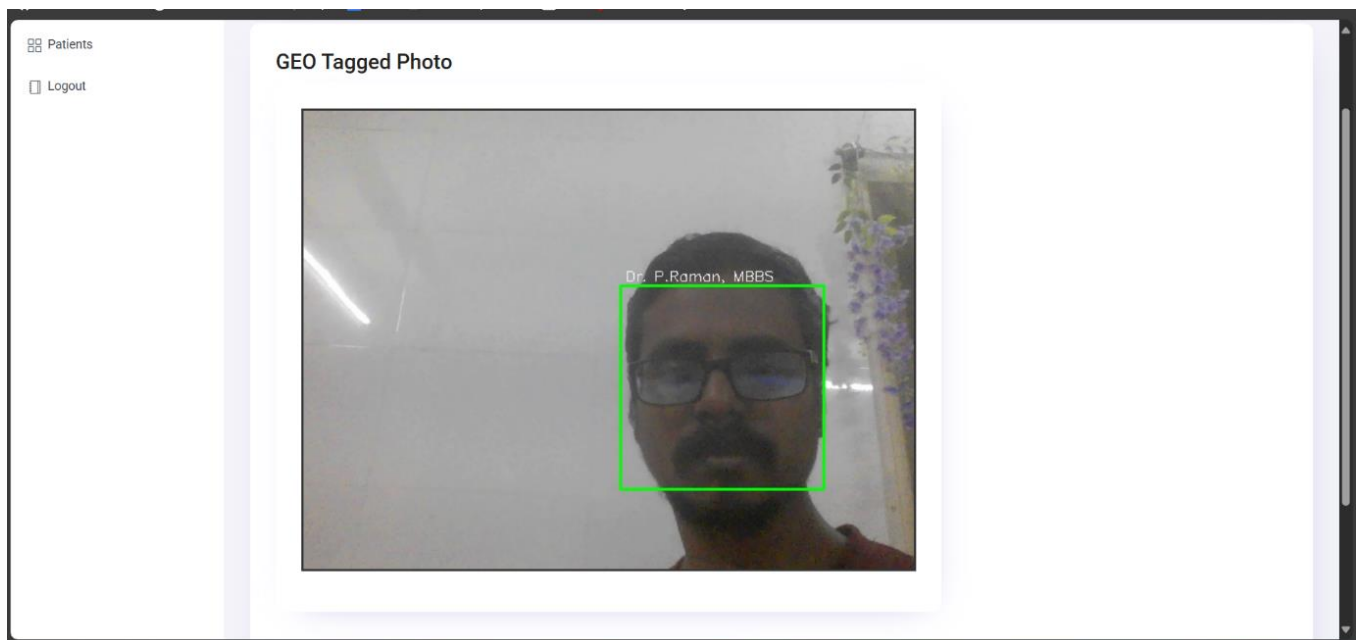
DI

LOGIN

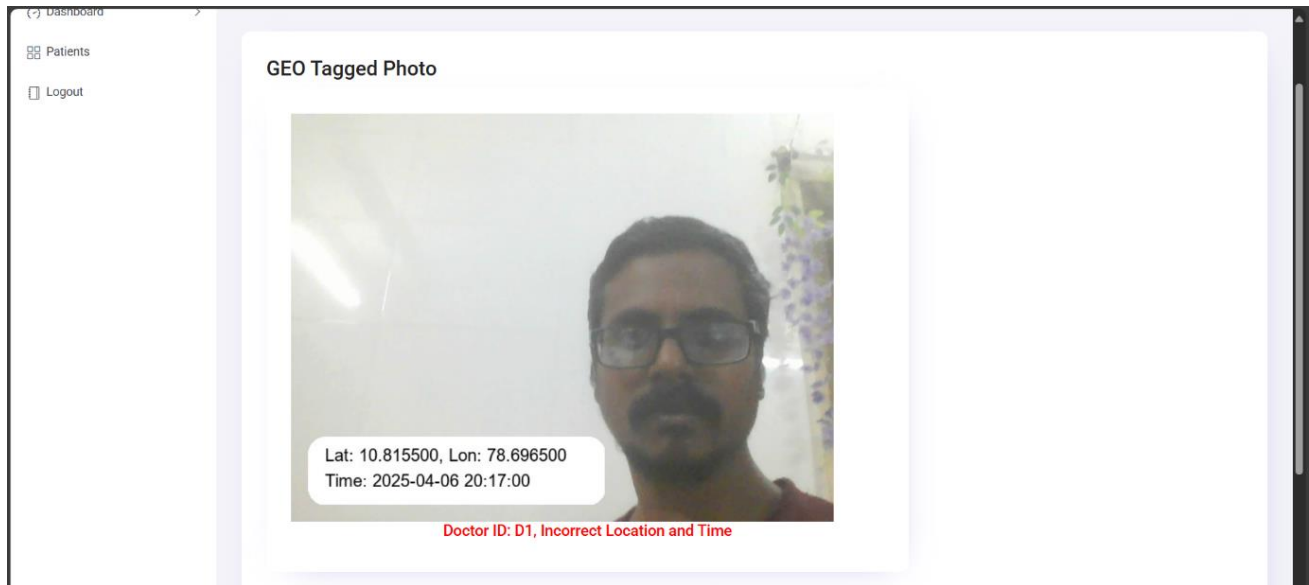
Doctor Login Page



Doctor ID and Attendance Capture



GEO Tagged Photo



GEO Tagged Photo

The screenshot shows a web application interface with a sidebar on the left containing 'Patients' and 'Logout'. The main content area is titled 'Add Patient Information' and contains a form with the following fields: 'Name' (filled with 'Raj'), 'Gender' (dropdown menu showing 'Male'), 'Age' (filled with '45'), 'Phone Number' (filled with '23585655588'), 'Health Disease' (filled with 'Fever'), 'Symptoms' (filled with 'cold, pain'), and 'Prescription' (empty text area). A green 'Add' button is located at the bottom of the form.

Add Patient Information

Dashboard
Add Patient Details

Patients
Logout

Patient Information

Patient: Raji		Date: 2025-04-06
Gender: Female	Age: 45	
Aadhar	: 235856555588	
Health Disease	: Fever	
Symptoms	" cold, pain	
Prescription	: Aspirin - 100mg, Morning-Night, for 3 days	

Patient Information

PHC Doctor Attendance
HOME DOCTOR DDHS HMO PHC ADMIN

DDHS Login

DDHS Login

DDHS

Dashboard
Monitor Attendance
Report
Logout

Monitor Attendance

P001-GH1-RR
06-04-2025
Go

Doctor ID: D1

Present



Time10:05:00
GPS Location10.815500, 78.696500
FaceVerified

Doctor ID: D2

Absent



Time
GPS Location
FaceNot Verified

Monitor Attendance

Dashboard

Monitor Attendance
Report
Logout

Doctor Report

P001-GH1-RR
-Month-
-Year-
Go

Doctor ID: D1



Doctor

Dr. P.Raman, MBBS
General Physician

Mobile No.

9874522555

Email

raman@gmail.com

No. of Days Present

3

No. of Days Absent

3

Attendance %

50.0

View Attendance / View Patients

Doctor ID: D2



Doctor

Dr. R.Geetha, M.D
General Physician

Mobile No.

8956222541

Email

geetha@gmail.com

No. of Days Present

0

No. of Days Absent

1

Attendance %

0.0

Doctor Report

70

Attendance Report: D1

Doctor ID: D1		2025-04-01- Absent
	Time	✗ 06:59:08
	GPS Location	✗ 06:59:08, 78.696500
	Face	✗ Not Verified
Doctor ID: D1		2025-04-02- Absent
	Time	✗ 08:51:54
	GPS Location	✓ 10.861600, 78.647000
	Face	✓ Verified
Doctor ID: D1		2025-04-03- Absent
	Time	✗ 11:18:04
	GPS Location	✗ 11:18:04, 78.696500
	Face	✓ Verified
Doctor ID: D1		2025-04-04- Present
	Time	✓ 11:19:52
	GPS Location	✓ 10.815500, 78.696500
	Face	✓ Verified
Doctor ID: D1		2025-04-05- Present
	Time	✓ 11:22:07
	GPS Location	✓ 10.815500, 78.696500

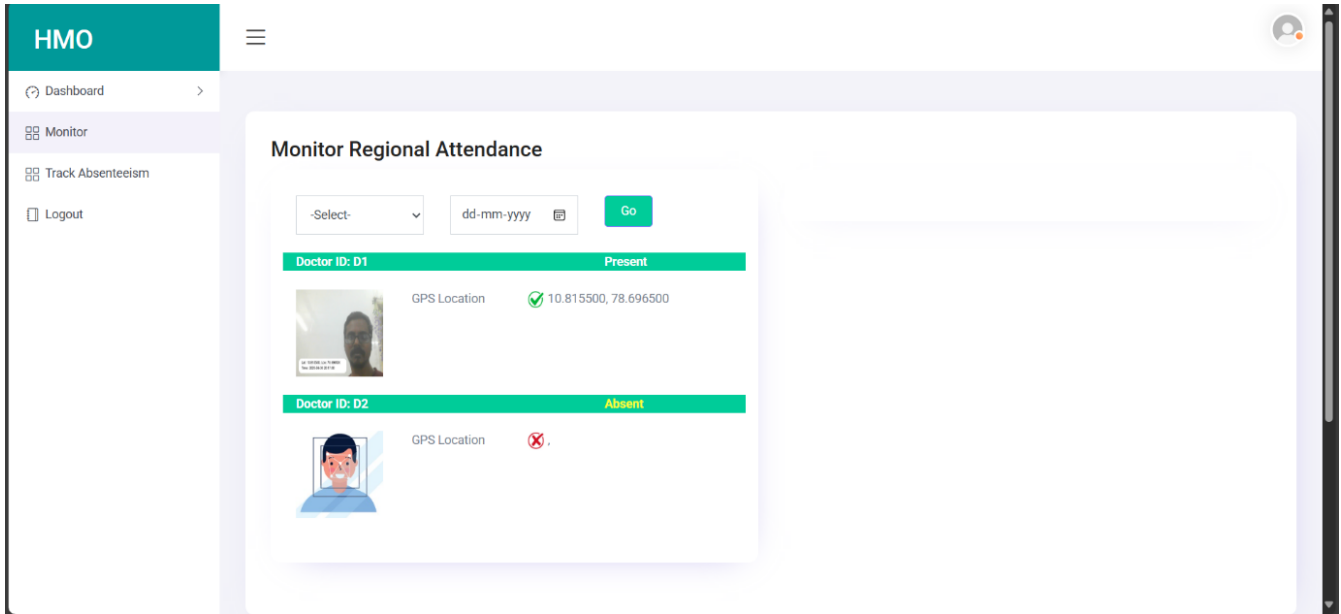
Attendance Report



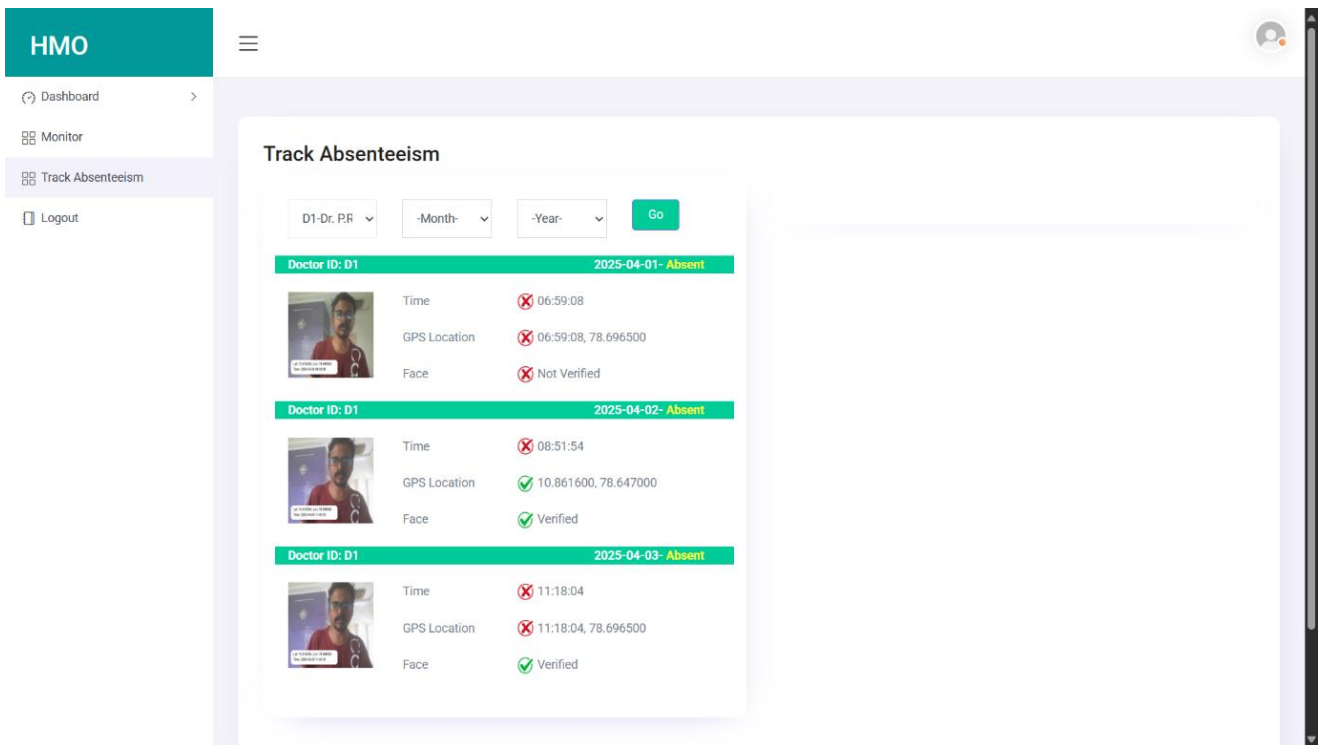
HMO Login

LOGIN

HMO Login



Monitor Regional Attendance



Track Absenteeism

BIBLIOGRAPHY

REFERENCE

1. M. A. Thalor and O. S. Gaikwad, "Facial Recognition Attendance Monitoring System using Deep Learning Techniques," *Int. J. Integr. Sci. Technol.*, vol. 1, no. 6, pp. 841–848, Jan. 2024.
2. Y. Kim and C. B. Soh, "A Hybrid Mobile-based Patient Location Tracking System for Personal Healthcare Applications," in *Proc. IEEE Eng. Med. Biol. Soc. (EMBC)*, 2022, pp. 1234–1237.
3. J. Smith and A. Doe, "Development and Implementation of a GPS-based Attendance Tracking System," *Int. J. Inf. Technol.*, vol. 10, no. 3, pp. 123–135, Jan. 2022.
4. C. Lee and S. Kim, "Development of an Attendance Management System using GPS and Mobile Application," *J. Inf. Process. Syst.*, vol. 16, no. 4, pp. 895–906, Jan. 2020.
5. R. Gupta, S. Sharma, and A. Singh, "Design and Implementation of Attendance Management System using GPS and RFID Technology," *Int. J. Eng. Res. Technol.*, vol. 8, no. 3, pp. 125–130, Jan. 2019.
6. K. Patel and R. Patel, "GPS and GSM based Student Tracking System," *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol.*, vol. 3, no. 3, pp. 270–276, Jan. 2018.
7. Y. Zhang, Y. Liu, and H. Li, "Design of a GPS-based Attendance System for University Classroom," *J. Phys.: Conf. Ser.*, vol. 890, no. 1, p. 012122, Jan. 2017.
8. L. Wang, Y. Huang, and H. Zhang, "Research on GPS-based Attendance Management System for University," in *Proc. Int. Conf. Educ., Manage. Sci. Econ. (ICEMSE)*, 2016, pp. 86–90.
9. Q. Zhang, W. Chen, and J. Zhang, "Design of GPS-based Attendance Management System," in *Proc. Int. Conf. Intell. Transp., Big Data Smart City (ICITBS)*, 2015, pp. 176–181.
10. Y. Wang and S. Li, "Design and Implementation of GPS-based Personnel Attendance System," in *Proc. Int. Conf. Comput. Sci. Softw. Eng. (ICCSSE)*, 2014, pp. 572–575.

BOOK REFERENCES

- "Flask Web Development" by Miguel Grinberg
- "Python Crash Course" by Eric Matthes
- "Learning MySQL" by Seyed M.M. Tahaghoghi and Hugh E. Williams
- "Bootstrap 4 Quick Start" by Jacob Lett
- "Python for Data Analysis" by Wes McKinney
- "Scikit-Learn Cookbook" by Trent Hauck

- "Data Visualization with Matplotlib and Seaborn" by Albert S. Lukaszewski
- "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron
- "Deep Learning with Python" by François Chollet
- "HTML and CSS: Design and Build Websites" by Jon Duckett
- "Fluent Python" by Luciano Ramalho
- "Python Data Science Handbook" by Jake VanderPlas

WEB REFERENCES

- Flask Web Development - <https://flask.palletsprojects.com>
- Python Crash Course (2nd ed.) - <https://nostarch.com/pythoncrashcourse2e>
- Learning MySQL - <https://www.oreilly.com/library/view/learning-mysql/>
- Bootstrap 4 Quick Start - <https://bootstrapcreative.com/bootstrap-4-quick-start-book/>
- Python for Data Analysis (3rd ed.) - <https://wesmckinney.com/book/>
- Scikit-Learn Cookbook - <https://www.packtpub.com/product/scikit-learn->
- Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.) - <https://www.oreilly.com/library/view>
- Deep Learning with Python (2nd ed.) - <https://www.manning.com/books/deep-learning-with-python-second-edition>
- HTML and CSS: Design and Build Websites - <https://www.htmlandcssbook.com/>
- Fluent Python (2nd ed.) - <https://www.oreilly.com/library/view/fluent-python-2nd/9781492056348/>
- Python Data Science Handbook - <https://jakevdp.github.io/PythonDataScienceHandbook/>